从 C++到 AI 计算：第三册实践卷

本地量化推理引擎、KV Cache、后端优化与验收

CPU Performance Study

本卷是《从 C++ 到 AI 计算：第三册》的配套实践卷，围绕 `linux_cpu_inference` 贯穿项目展开：模型资产、tokenizer、张量布局、int8 kernel、reference decoder、KV cache、trace、7B 账本、服务调度、CPU/GPU 后端边界和工程验收都要落到可构建代码、测试和报告。

前言

第三册原理卷把 AI 推理系统拆成模型资产、typed graph、张量布局、量化、KV cache、后端、服务运行时和验收体系。本实践卷只做一件事：把这些概念压到同一个可运行项目里，让读者知道每个概念对应哪段代码、哪条命令、哪种输入、哪份报告和哪条失败路径。

贯穿项目是 `books/cpu-volume-3/labs/linux_cpu_inference`。它不是玩具目录，而是第三册已经持续维护的本地 CPU 量化推理切片：包含教学模型格式、tokenizer、safetensors manifest gate、int8 linear、packed/AVX2 路径、tiny reference decoder、KV cache trace、7B compute ledger、serving SLO probe、CLI、benchmark 和 CTest。实践卷不复制这套代码，避免两份实现漂移；所有练习都回到同一份源码。

本卷的学习顺序是从小证据到大系统：先让一个 tiny MLP 算对，再解释张量地址流和 int8 累加；再让 reference decoder、sampler 和 KV cache trace 固定语义；然后接模型导入、shape verifier、memory planner 和 IR lowering；最后用 CPU/GPU 后端边界、工业专项和回归门禁收束。每章都要求读者交付代码运行结果或报告字段，而不是只抄概念定义。

核心思想

> 第三册实践的目标不是“写一个看起来像大模型的 demo”，而是建立一条能被复查的推理证据链：资产能校验，shape 能验证，kernel 能对照，trace 能定位，性能能降级，服务指标能解释，回归门禁能长期维护。

常见缩写和术语先导

缩写	全称	本卷中的含义
AI	Artificial Intelligence	这里特指本地推理工程，不讨论泛泛应用口号。
LLM	Large Language Model	decoder-only 文本生成模型是本卷主线。
KV Cache	Key/Value Cache	attention 历史 K/V 状态，属于请求生命周期。
GEMM	General Matrix Multiply	矩阵乘，是预填充和训练常见核心算子。
GEMV	General Matrix Vector Multiply	矩阵向量乘，是单 token decode 的核心形态。
SIMD	Single Instruction Multiple Data	一条指令处理多个 lane 的 CPU 执行方式。
AVX2	Advanced Vector Extensions 2	x86-64 上的 256-bit SIMD 指令集之一。
FMA	Fused Multiply-Add	乘加融合指令，常出现在浮点 kernel 中。
SLO	Service Level Objective	服务目标，例如 TTFT、TPOT、p95/p99。
TTFT	Time To First Token	从请求进入到首 token 输出的时间。
TPOT	Time Per Output Token	decode 阶段每个输出 token 的平均或尾部时间。
IR	Intermediate Representation	编译器内部表示，用于 pass、lowering 和计划生成。
RAG	Retrieval-Augmented Generation	检索增强生成，需要把外部文档和 prompt 编排起来。
MoE	Mixture of Experts	多专家模型，会改变路由、负载和内存账本。
LoRA	Low-Rank Adaptation	低秩适配权重，常用于微调和多租户加载。

这些缩写不要死记。每个缩写在正文里都要落到一个代码对象或报告字段：KV cache 对应 trace 里的 slot 和 position，SLO 对应 serving probe 的 CSV，IR 对应 executable plan，SIMD/AVX2 对应 kernel 反汇编和 benchmark。

本卷结构

本卷按第三册原书的四个大块组织。每一章都对应原书的一组主题文件，并把概念落到同一条实践主线：`linux_cpu_inference`。

第一部分从模型资产到量化 kernel。你会运行 tiny MLP，检查 tokenizer 和 safetensors manifest，手算张量地址，比较 scalar、packed 和 AVX2 路径，并解释 int8 scale、accumulator 和 requantize 的边界。

第二部分进入 Transformer、KV cache、sampler 和服务运行时。你会跑 reference decoder trace，解释每个 checkpoint 的 shape、stride、dtype 和 layout，构造 KV cache 状态表，分析 admission、queue wait、TTFT、TPOT、取消和拒绝。

第三部分处理模型导入、真实模型加载闭环、memory planner 和 IR lowering。你会把 manifest、shape verifier、tensor role、state edge、workspace、liveness 和 executable plan 写成可检查报告。

第四部分收束到后端优化和验收。你会比较 CPU kernel 证据、GPU 后端边界、工业专项能力和回归门禁。最终交付不是“跑通一次”，而是可复查的工程档案：命令、输入、输出、trace、benchmark、不可验证边界和回归规则。

目录

第零部分

推理链路、张量账本与量化

第 1 章实践：端到端推理链路与项目总入口

本章对应原书中的第三册“端到端链路：从模型文件到下一个 token”。不要把它当作概念复述，本章要把模型目录、manifest、tokenizer、typed graph、backend plan、request session、KV cache、oracle 和 profiler 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `tiny_mlp_i8.txt`，核心命令或测试入口是 `lcqi_cli`。

Listing 1.1 第 1 章实践：端到端推理链路与项目总入口的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / tiny_mlp_i8.txt
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：只跑一个 tiny MLP 推理请求。读者要先手写预期结果，再运行项目观察输出。动作是：把 `readfiles->tokenize->run_inference->logits->predicted_class` 串起来。如果输出里没有 `predicted_class` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型文件存在不等于推理链路可信；只有 CLI 输出、测试断言、输入合同和失败路径同时成立，才算完成入口闭环这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：最终大作业的总目录：每章报告都放入 `reports/volume3-practice/`，第一章提交项目地图、命令记录和一次成功/失败输入对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 2 章实践：张量布局、地址流与第一个 MatVec

本章对应原书中的第三册“张量布局、地址流与第一个矩阵向量内核”。不要把它当作概念复述，本章要把 TensorView、dtype、shape、stride、row-major、地址公式、bias、ReLU、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `linear_i8`，核心命令或测试入口是 `lcqi_tests`。

Listing 2.1 第 2 章实践：张量布局、地址流与第一个 MatVec 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / linear_i8
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：手算一个 2 行 4 列权重矩阵的地址偏移。读者要先手写预期结果，再运行项目观察输出。动作是：把 `weight[out,k]` 展开成 `base+(out*K+k)*sizeof(i8)`，再对应到 `linear_i8` 的循环。如果输出里没有 `logit` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：张量不是裸数组；每个 kernel 入口都要能解释 dtype、shape、stride、scale 和输出缓冲这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一页地址流账本：输入向量、hidden 权重、输出 logits 各自的 shape、连续性、读写次数和错误边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 3 章实践：GEMM、packing 与 SIMD 证据

本章对应原书中的第三册“从矩阵向量乘走向 GEMM、packing 与 SIMD”。不要把它当作概念复述，本章要把 packed weight、scalar baseline、AVX2 backend、output block、shape sweep、反汇编和降级放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `output_block`，核心命令或测试入口是 `lcqi_bench`。

Listing 3.1 第 3 章实践：GEMM、packing 与 SIMD 证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / output_block
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：同一组输入同时跑 `scalar`、`packed_scalar` 和 `avx2`。读者要先手写预期结果，再运行项目观察输出。动作是：比较 CSV 中 `backend`、`available`、`average_us`、`max_abs_diff` 和 `checksum`。如果输出里没有 `max_abs_diff` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：SIMD 路径只有在结果和 baseline 对齐后才有资格谈速度；`available=0` 也是证据，不是失败这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 shape sweep 表，至少解释一个 SIMD 有收益、一个收益不明显、一个当前环境不可验证的场景。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 4 章实践：int8 量化、累加器与重新量化

本章对应原书中的第三册“量化系统总览”和“int8 量化、累加器与重新量化”。不要把它当作概念复述，本章要把 int8 weight、float activation、scale、accumulator、requantize、oracle tolerance 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `TinyMlpModel`，核心命令或测试入口是 `lcqi_tests`。

Listing 4.1 第 4 章实践：int8 量化、累加器与重新量化的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / TinyMlpModel
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：构造一行 int8 权重 `[2,-1,3,0]` 和输入 `[1,2,-1,0.5]`。读者要先手写预期结果，再运行项目观察输出。动作是：先手算 int32/float accumulator，再和 `linear_i8` 输出比较。如果输出里没有 `scale` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：量化不是把 float 强转成 int8；必须记录 scale 作用位置、累加宽度、误差阈值和 golden 输出这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交量化合同卡：每个 tensor 的 dtype、scale 粒度、accumulator 类型、误差门限和拒绝条件。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

Transformer、KV Cache 与服务运行时

第 5 章实践：Transformer 切片与 KV Cache 状态

本章对应原书中的第三册“AI 推理原理、KV cache 与计算账本”和“KV cache 实现细节”。不要把它当作概念复述，本章要把 RMSNorm、Q/K/V、RoPE、GQA、KV slot、position、state edge 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `kv_cache`，核心命令或测试入口是 `lcqi_trace`。

Listing 5.1 第 5 章实践：Transformer 切片与 KV Cache 状态的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / kv_cache
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：固定 prompt `tok1tok2tok3`，观察第一个 decode step。读者要先手写预期结果，再运行项目观察输出。动作是：把 trace 中 tokenizer、q/k/v、`kv_cache_slot`、attention 和 logits 串成状态表。如果输出里没有 `token_position` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：KV cache 是请求状态，不是临时 activation；position、layer、head、layout 和生命周期必须进入 trace 这条规律要写进报告，不要只写 “运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为 “接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 KV cache 状态表：每层每个 token 写入哪里、何时读取、何时禁止复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 6 章实践：Reference Decoder、logits 与采样

本章对应原书中的第三册“Reference decoder 实现”和“推理算法：logits、softmax、采样”。不要把它当作概念复述，本章要把 reference decoder、logits golden、argmax sampler、tokenizer contract hash、BOS/EOS 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `argmax`，核心命令或测试入口是 `lcqi_trace`。

Listing 6.1 第 6 章实践：Reference Decoder、logits 与采样的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / argmax
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：让 trace 固定输出 tokenizer contract 和 logits golden。读者要先手写预期结果，再运行项目观察输出。动作是：比较 generated token、logits 数组和 tokenizer hash。如果输出里没有 `tokenizer_contract` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：采样前必须先证明 logits 可复查；sampler 的随机性、温度和 top-k 只能建立在 reference 通过之后这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交采样报告：argmax、top-k、temperature 三种策略各自需要哪些额外字段才能复现。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 7 章实践：服务调度、SLO 与资源预算

本章对应原书中的第三册“业务服务实战：请求、调度、队列、取消与资源预算”。不要把它当作概念复述，本章要把 admission、KV budget、queue wait、TTFT、TPOT、cancel、reject、p95 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `ttft_ms`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 7.1 第 7 章实践：服务调度、SLO 与资源预算的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / ttft_ms
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：运行 serving probe 中短请求、RAG 请求、取消请求和超长请求。读者要先手写预期结果，再运行项目观察输出。动作是：解释 `memory_budget_exceeded`、cancel reclaim、`queue_wait_p95_ms` 和 `rejection_rate`。如果输出里没有 `rejection_rate` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：推理服务不是只看 tokens/s；是否接收请求、如何预留 KV、取消后是否回收，决定系统是否可运营这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 SLO 表：每类请求的 prompt/new token、预算、是否接受、TTFT、TPOT、拒绝原因和未验证边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 8 章实践：RAG、Agent 与状态图边界

本章对应原书中的第三册“上层 AI 应用编排：RAG、Agent、工具和状态图”。不要把它当作概念复述，本章要把 retrieval、tool call、prompt assembly、state graph、budget、trace id 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `req_rag`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 8.1 第 8 章实践：RAG、Agent 与状态图边界的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / req_rag
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 RAG 请求拆成 retrieve、rerank、prompt build、prefill、decode 五段。读者要先手写预期结果，再运行项目观察输出。动作是：为每段写输入输出和失败后能否重试。如果输出里没有 `prompt_tokens` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：上层编排不能把外部工具结果和模型状态混成字符串；每个边界都要有身份、预算和可重放记录这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 RAG 状态图：节点、边、超时、重试、缓存键、prompt 版本和审计字段。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

模型导入、AI 编译器与内存计划

第 9 章实践：模型导入、manifest 与 Shape Verifier

本章对应原书中的第三册“模型导入、manifest 与 shape verifier”。不要把它当作概念复述，本章要把 safetensors header、metadata、tensor name、dtype、shape、`data_offsets`、payload boundary 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `load_safetensors_manifest`，核心命令或测试入口是 `lcqi_tests`。

Listing 9.1 第 9 章实践：模型导入、manifest 与 Shape Verifier 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / load_safetensors_manifest
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：加载测试临时 safetensors fixture。读者要先手写预期结果，再运行项目观察输出。动作是：故意构造 offset 越界和 dtype/shape 字节数不匹配。如果输出里没有 `data_offsets` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型导入的第一职责是早拒绝：名字、shape、dtype、offset 和 payload 都必须在 kernel 运行前被验证这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 manifest gate 报告：合法 fixture 和两个坏 fixture 的字段、错误原因和拒绝位置。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 10 章实践：真实模型加载闭环与首 Token Trace

本章对应原书中的第三册“真实模型加载闭环：Tokenizer、权重格式与首 token trace”。不要把它当作概念复述，本章要把 tokenizer、weight mapping、chat template、first token trace、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `tiny_tokenizer.txt`，核心命令或测试入口是 `lcqi_trace`。

Listing 10.1 第 10 章实践：真实模型加载闭环与首 Token Trace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / tiny_tokenizer.txt
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：从 tokenizer fixture 开始，不跳到真实 7B。读者要先手写预期结果，再运行项目观察输出。动作是：解释 BOS/EOS、UNK、template hash 和 decoder 输入 tokens 的边界。如果输出里没有 `tokens=[0,1,2,3,4]` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：真实模型加载不是下载权重；必须先让 tokenizer 合同、权重 role、shape verifier 和首 token trace 形成闭环这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交真实模型迁移清单：需要新增哪些 tokenizer、name mapping、quant descriptor 和 golden trace。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 11 章实践：运行时内存计划、liveness 与 Workspace

本章对应原书中的第三册“推理运行时与内存规划”。不要把它当作概念复述，本章要把 activation arena、workspace、liveness、state edge、debug dump、buffer reuse 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `layout`，核心命令或测试入口是 `lcqi_trace`。

Listing 11.1 第 11 章实践：运行时内存计划、liveness 与 Workspace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / layout
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：比较普通 activation 和 KV cache 的生命周期。读者要先手写预期结果，再运行项目观察输出。动作是：说明为什么 KV cache 不能被 activation planner 当作临时缓冲复用。如果输出里没有 `checksum` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：内存复用只在生命周期不重叠时成立；debug/oracle/materialization 会延长生命周期这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 liveness 表：每个 checkpoint 的 `defined_at`、`last_used_at`、是否 state、是否允许复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 12 章实践：IR、Pass、Lowering 与 Executable Plan

本章对应原书中的第三册“AI 编译器细化：IR、pass、lowering 与 executable plan”。不要把它当作概念复述，本章要把 typed graph、pass、fusion、layout propagation、backend capability、fallback reason 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `run_inference`，核心命令或测试入口是 `lcqi_cli`。

Listing 12.1 第 12 章实践：IR、Pass、Lowering 与 Executable Plan 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / run_inference
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 tiny MLP 写成三节点 IR：linear、relu、linear。读者要先手写预期结果，再运行项目观察输出。动作是：说明哪些 pass 可以融合，哪些必须保留为 oracle checkpoint。如果输出里没有 `backend` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：IR 的价值不是画图，而是让 shape、layout、量化、后端选择和 fallback 诊断有稳定承载物这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 executable plan 草图：节点、输入输出 tensor、layout、kernel、fallback reason 和 profiler event。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

后端优化、工业专项与验收

第 13 章实践：CPU 后端、微内核与性能证据

本章对应原书中的第三册“CPU 后端优化细讲”和“CPU decode 实战”。不要把它当作概念复述，本章要把 scalar baseline、packed weight、AVX2、objdump、perf boundary、shape sweep 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `avx2`，核心命令或测试入口是 `lcqi_bench`。

Listing 13.1 第 13 章实践：CPU 后端、微内核与性能证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / avx2
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：对同一 kernel 保存 benchmark CSV 和反汇编摘要。读者要先手写预期结果，再运行项目观察输出。动作是：解释速度差来自地址流、SIMD lane、packing、load 或 reduction 哪一项。如果输出里没有 `average_us` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：CPU 后端优化必须同时有 oracle、反汇编、shape sweep 和降级边界；只有耗时表不够这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 CPU kernel evidence：命令、机器、编译器、输入 shape、backend、checksum、反汇编摘要和未验证 PMU 边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 14 章实践：GPU 后端边界、copy 与同步成本

本章对应原书中的第三册“GPU 硬件基础”和“GPU 后端优化细讲”。不要把它当作概念复述，本章要把 device buffer、stream、event、kernel launch、copy、sync、fallback 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `StepTrace`，核心命令或测试入口是 `lcqi_trace`。

Listing 14.1 第 14 章实践：GPU 后端边界、copy 与同步成本的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / StepTrace
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：当前项目没有 GPU kernel，也要写清 adapter 合同。读者要先手写预期结果，再运行项目观察输出。动作是：把 CPU trace 字段扩展成 GPU StepTrace 需要哪些字段。如果输出里没有 `copy_time_ns` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：GPU 快不快不能只看 kernel；copy、sync、sampler、fallback 和 batch wait 必须进入端到端 trace 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 GPU adapter 设计：buffer owner、stream、event、H2D/D2H 字节、sync 点、fallback 和 oracle 对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 15 章实践：工业 LLM 专项能力对照

本章对应原书中的第三册“工业 LLM 专项：投机解码、MoE、LoRA、多 GPU 与源码对照”。不要把它当作概念复述，本章要把 speculative decoding、MoE routing、LoRA adapter、multi-GPU、source reading 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `bandwidth_limit`，核心命令或测试入口是 `lcqi_ledger`。

Listing 15.1 第 15 章实践：工业 LLM 专项能力对照的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_ledger / bandwidth_limit
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：选择一个专项，不写泛泛介绍。读者要先手写预期结果，再运行项目观察输出。动作是：把它接到现有 trace、memory、backend 或 serving 字段上。如果输出里没有 `tokens/s` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：工业专项不是功能清单；必须说明它购买什么能力、增加什么状态、破坏哪些假设这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交专项设计卡：动机、状态新增、trace 字段、回归测试、性能账本和不能验证的部分。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 16 章实践：工程验收、回归门禁与最终项目

本章对应原书中的第三册“工程验收与回归体系”。不要把它当作概念复述，本章要把 reference、unit test、trace golden、benchmark gate、SLO gate、coverage、CI 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `lcqi_tests`，核心命令或测试入口是 `ctest`。

Listing 16.1 第 16 章实践：工程验收、回归门禁与最终项目的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: ctest / lcqi_tests
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把所有前面报告合成最终验收包。读者要先手写预期结果，再运行项目观察输出。动作是：运行 `ctest` 并说明每个 test 保护哪条合同。如果输出里没有 `100% tests passed` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：实践卷最终目标是长期维护：结果正确、trace 稳定、性能可解释、服务指标可回归、失败边界不伪装这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交最终项目：README、命令记录、报告索引、测试结果、benchmark 表、trace 摘要、风险清单和下一阶段计划。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

完整源码清单

本附录从 books/cpu-volume-3/labs/linux_cpu_inference 直接读入贯穿项目源码。实践卷正文只解释每章要看哪一段；真正的闭环必须回到完整工程：头文件、实现、CLI、benchmark、trace、测试和模型 fixture 都要能一起构建。

一、构建入口和项目说明

Listing 16.2 第三册 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.31)
project(CPUVolume3 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

option(CPU_VOLUME_3_BUILD_LABS "Build CPU Volume 3 labs" ON)

if(CPU_VOLUME_3_BUILD_LABS)
    enable_testing()
    add_subdirectory(labs/linux_cpu_inference)
endif()
```

Listing 16.3 linux_cpu_inference CMakeLists.txt

```
add_library(lcqi STATIC
    src/inference.cpp
    src/int8_kernels.cpp
    src/model.cpp
    src/reference_decoder.cpp
    src/safetensors.cpp
    src/tokenizer.cpp
)

if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
    if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
        target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
        set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES ↩
    ↪ COMPILE_OPTIONS "-mavx2;-mfma")
        target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
    endif()
endif()

if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
    message(WARNING "LCQI is written for the book's local Linux CPU target; ↩
    ↪ current system is ${CMAKE_SYSTEM_NAME}.")
endif()
```

```
21
22 target_include_directories(lcqi
23     PUBLIC
24         ${CMAKE_CURRENT_SOURCE_DIR}/include
25 )
26
27 target_compile_features(lcqi PUBLIC cxx_std_20)
28
29 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
30     target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
31 endif()
32
33 add_executable(lcqi_cli
34     src/main.cpp
35 )
36
37 target_link_libraries(lcqi_cli PRIVATE lcqi)
38
39 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
40     target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
41 endif()
42
43 add_executable(lcqi_bench
44     src/bench.cpp
45 )
46
47 target_link_libraries(lcqi_bench PRIVATE lcqi)
48
49 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
50     target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
51 endif()
52
53 add_executable(lcqi_trace
54     src/trace.cpp
55 )
56
57 target_link_libraries(lcqi_trace PRIVATE lcqi)
58
59 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
60     target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
61 endif()
62
63 add_executable(lcqi_ledger
64     src/ledger.cpp
65 )
66
67 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
68     target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
69 endif()
70
71 add_executable(lcqi_serving_probe
72     src/serving_probe.cpp
```

```
)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
endif()

find_package(dnnl QUIET)
if(dnnl_FOUND)
    add_executable(lcqi_onednn_bench
        src/onednn_bench.cpp
    )
    target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
    target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
    if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
        target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra -Wpedantic)
    endif()
endif()

add_executable(lcqi_tests
    tests/lcqi_tests.cpp
)

target_link_libraries(lcqi_tests PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_test(NAME lcqi_tests COMMAND $<TARGET_FILE:lcqi_tests>)
add_test(NAME lcqi_trace COMMAND $<TARGET_FILE:lcqi_trace>)
set_tests_properties(lcqi_trace PROPERTIES
    PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
)
add_test(NAME lcqi_ledger COMMAND $<TARGET_FILE:lcqi_ledger>)
set_tests_properties(lcqi_ledger PROPERTIES
    PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
)
add_test(NAME lcqi_serving_probe COMMAND $<TARGET_FILE:lcqi_serving_probe>)
set_tests_properties(lcqi_serving_probe PROPERTIES
    PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
)
```

Listing 16.4 linux_cpu_inference README.md

```
# Linux CPU Quantized Inference

这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache、CPU/GPU 后端、正确性报告和性能对标。

```

目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变当前切片的语义合同。

当前版本支持：

- 教学文本模型格式 `LCQI_MODEL_V1`
- 最小 tokenizer 合同格式 `LCQI_TOKENIZER_V1`
- safetensors header manifest 解析：metadata、tensor name、dtype、shape、data offsets、offset 边界和 dtype/shape 字节数校验
- 两层 MLP 教学切片
- int8 权重加 per-layer scale
- float 输入和激活
- 量化线性层、ReLU、分类 argmax
- int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
- tiny reference decoder：RMSNorm、float linear、RoPE、GQA KV cache、decode attention、SwiGLU、lm head
- reference trace CSV：从 prompt/tokenizer 合同到 logits/sampler/token，逐 checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values 摘要
- 7B ledger CSV：输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限
- serving SLO probe CSV：输出 admission、batch state、KV 预算、queue wait、TTFT、TPOT、取消回收和拒绝率
- CLI 推理和简单 benchmark
- CTest 正确性测试

后续扩展方向：

- 真实 7B 级模型 metadata 和 tokenizer
- 完整 BPE/SentencePiece tokenizer、GGUF/safetensors 权重导入、分片加载和 name mapping
- 多层 decoder-only Transformer reference path
- KV cache 分页、内存规划和可选量化
- CPU packed weight、SIMD、线程和 NUMA 优化
- GPU backend adapter 和 device kernel
- 与现有框架的 prefill/decode/内存/误差对标报告

构建：

```bash
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
```

运行：

```bash
```

```
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
  books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
  1.0 2.0 -1.0 0.5 1000
```

kernel benchmark：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
```

输出 CSV 字段：

```text
target_arch,input_size,output_size,output_block,repeat,backend,available,average_us,max_abs_diff,checksum
```

当前 benchmark 按 backend 逐行输出：`scalar`、`packed_scalar`、`avx2`。x86-64 构建会编译 AVX2 路径；不支持 AVX2/FMA 的机器会把该 backend 标为 `available=0`，避免把源码存在误报成当前环境实测性能。

这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据，还必须继续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对照、sanitizer、fuzz 和 CI。

safetensors manifest gate：

`lcqi_tests` 会运行时生成一个最小 safetensors fixture，验证 header 长度、`__metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape 字节数不匹配的坏文件，确认加载器会拒绝。这个 gate 只覆盖模型资产目录表，不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行；后续真实模型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant descriptor 和 first token trace。

reference decoder trace：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
```

输出 CSV 字段：

```text
name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
```

这条 trace 固定 prompt fixture `tok1 tok2 tok3`，tokenizer 输出完整 `tokens=[0,1,2,3,4]`，其中 `0/4` 是 BOS/EOS；decoder trace 会剥离 BOS/EOS 后进入 tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同，避免

把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`lcqi_tests` 会检查 tokenizer contract hash、关键 checkpoint 的 shape、stride、dtype、layout 和 logits golden，CTest 也会直接运行 `lcqi_trace`。

当前 tokenizer fixture 的关键输出：

```text
tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-chat-template-v1
```

7B ledger：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
  > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
```

输出 CSV 字段：

```text
section,item,value,unit,notes
```

这份账本固定 `L=32,H=4096,n_q=32,n_kv=8,head_dim=128,context=4096`，报告 decode FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s 上限。CTest 会检查 `int4_at_100_gbps=23.602 tokens/s` 这一行，防止账本公式漂移。

serving SLO probe：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe \
  > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
```

输出 CSV 字段：

```text
row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,metric_value
```

probe 固定四个请求：短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按 `prompt_tokens + max_new_tokens` 预留 KV，取消请求释放预算，超长请求返回 `memory_budget_exceeded`，并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、`

```
    ↪ rejection_rate` 等服务化指标。
```

二、公共合同头文件

Listing 16.5 include/lcqi/inference.hpp

```cpp
#pragma once

#include <cstdint>
#include <span>
#include <vector>

#include <lcqi/model.hpp>

namespace lcqi {

struct InferenceResult {
    std::vector<float> logits;
    std::int32_t predicted_class = 0;
};

void linear_i8(const QuantizedLinearLayer& layer,
               std::span<const float> input,
               std::span<float> output);

void relu_inplace(std::span<float> values);

InferenceResult run_inference(const TinyMlpModel& model,
                              std::span<const float> input);

}  // namespace lcqi
```

Listing 16.6 include/lcqi/int8_kernels.hpp

```cpp
#pragma once

#include <cstdint>
#include <span>
#include <vector>

#include <lcqi/model.hpp>

namespace lcqi {

struct PackedLinearI8 {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
    float scale = 1.0F;
    std::vector<std::int8_t> packed_weights;
    std::vector<float> bias;
```

```
};

struct KernelBenchmarkCase {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
    std::int32_t repeat = 0;
};

struct KernelBackendBenchmark {
    const char* name = "";
    bool available = false;
    double average_us = 0.0;
    float max_abs_diff = 0.0F;
    float checksum = 0.0F;
};

struct KernelBenchmarkResult {
    KernelBenchmarkCase benchmark_case;
    std::vector<KernelBackendBenchmark> backends;
};

PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
                              std::int32_t output_block_size);

void linear_i8_packed(const PackedLinearI8& layer,
                      std::span<const float> input,
                      std::span<float> output);

void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
                                     std::span<const float> input,
                                     std::span<float> output,
                                     std::span<float> accumulator_workspace);

bool linear_i8_packed_avx2_available() noexcept;

void linear_i8_packed_avx2(const PackedLinearI8& layer,
                           std::span<const float> input,
                           std::span<float> output);

QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
                                                 std::int32_t output_size,
                                                 float scale);

std::vector<float> make_deterministic_input(std::int32_t input_size);

float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs);

KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ↩
    ↪ benchmark_case);

}  // namespace lcqi
```

Listing 16.7 include/lcqi/model.hpp

```hpp
#pragma once

#include <cstdint>
#include <filesystem>
#include <string>
#include <vector>

namespace lcqi {

struct QuantizedLinearLayer {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    float scale = 1.0F;
    std::vector<std::int8_t> weights;
    std::vector<float> bias;
};

struct TinyMlpModel {
    std::int32_t input_size = 0;
    std::int32_t hidden_size = 0;
    std::int32_t output_size = 0;
    QuantizedLinearLayer hidden;
    QuantizedLinearLayer output;
};

TinyMlpModel load_model(const std::filesystem::path& path);

}  // namespace lcqi
```

Listing 16.8 include/lcqi/reference_decoder.hpp

```hpp
#pragma once

#include <cstdint>
#include <span>
#include <string>
#include <vector>

namespace lcqi {

struct LinearWeightsF32 {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::vector<float> weights;
    std::vector<float> bias;
};

struct DecoderConfig {
    std::int32_t hidden_size = 0;
    std::int32_t query_heads = 0;
```

```
    std::int32_t kv_heads = 0;
    std::int32_t head_dim = 0;
    std::int32_t intermediate_size = 0;
    std::int32_t vocab_size = 0;
    std::int32_t max_context = 0;
    float rms_epsilon = 1.0e-5F;
    float rope_base = 10000.0F;
};

struct DecoderLayerWeightsF32 {
    std::vector<float> rms_attention_weight;
    LinearWeightsF32 wq;
    LinearWeightsF32 wk;
    LinearWeightsF32 wv;
    LinearWeightsF32 wo;
    std::vector<float> rms_mlp_weight;
    LinearWeightsF32 w_gate;
    LinearWeightsF32 w_up;
    LinearWeightsF32 w_down;
};

struct ReferenceDecoderModel {
    DecoderConfig config;
    std::vector<float> token_embedding;
    std::vector<DecoderLayerWeightsF32> layers;
    std::vector<float> final_norm_weight;
    LinearWeightsF32 lm_head;
};

struct ReferenceDecodeResult {
    std::vector<float> logits;
    std::int32_t predicted_token = 0;
};

struct ReferenceTensorCheckpoint {
    std::string name;
    std::int32_t token_position = 0;
    std::int32_t layer_id = 0;
    std::vector<std::int32_t> shape;
    std::vector<std::int32_t> stride;
    std::string dtype;
    std::string layout;
    float checksum = 0.0F;
    float max_abs = 0.0F;
    std::vector<float> values;
};

struct ReferenceDecodeTraceResult {
    ReferenceDecodeResult result;
    std::vector<ReferenceTensorCheckpoint> checkpoints;
};

```

```cpp
class ReferenceKVCache {
public:
    ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count);

    void append(std::int32_t layer_id,
                std::int32_t model_position,
                std::span<const float> key,
                std::span<const float> value);

    [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t kv_head) const;

    [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t kv_head) const;

    [[nodiscard]] std::int32_t layer_count() const noexcept;
    [[nodiscard]] std::int32_t filled_tokens() const noexcept;

private:
    [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t kv_head) const;
    void validate_address(std::int32_t layer_id,
                          std::int32_t model_position,
                          std::int32_t kv_head) const;

    DecoderConfig config_;
    std::int32_t layer_count_ = 0;
    std::int32_t filled_tokens_ = 0;
    std::vector<float> keys_;
    std::vector<float> values_;
    std::vector<std::uint8_t> written_;
};

void rms_norm(std::span<const float> input,
              std::span<const float> weight,
              float epsilon,
              std::span<float> output);

void linear_f32(const LinearWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output);

void apply_rope(std::span<float> heads,
                std::int32_t head_count,
                std::int32_t head_dim,
                std::int32_t model_position,
                float rope_base);

void attention_decode(const DecoderConfig& config,
```

```
                        const ReferenceKVCache& cache,
                        std::int32_t layer_id,
                        std::int32_t model_position,
                        std::span<const float> query,
                        std::span<float> output);

ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
                                           std::span<const std::int32_t> token_ids);

ReferenceDecodeTraceResult run_reference_decode_with_trace(
    const ReferenceDecoderModel& model,
    std::span<const std::int32_t> token_ids);

ReferenceDecoderModel make_tiny_reference_decoder_model();

}  // namespace lcqi
```

Listing 16.9 include/lcqi/safetensors.hpp

```
#pragma once

#include <cstdint>
#include <filesystem>
#include <string>
#include <string_view>
#include <utility>
#include <vector>

namespace lcqi {

struct SafeTensorEntry {
    std::string name;
    std::string dtype;
    std::vector<std::int64_t> shape;
    std::uint64_t data_begin = 0;
    std::uint64_t data_end = 0;

    [[nodiscard]] std::uint64_t byte_size() const;
};

struct SafeTensorManifest {
    std::uint64_t header_size = 0;
    std::uint64_t data_start_offset = 0;
    std::vector<std::pair<std::string, std::string>> metadata;
    std::vector<SafeTensorEntry> tensors;

    [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) const;
};

[[nodiscard]] SafeTensorManifest load_safetensors_manifest(
```

```
    const std::filesystem::path& path);

}  // namespace lcqi
```

Listing 16.10 include/lcqi/tokenizer.hpp

```
#pragma once

#include <cstdint>
#include <filesystem>
#include <string>
#include <string_view>
#include <vector>

namespace lcqi {

struct TokenEntry {
    std::string text;
    std::int32_t id = 0;
};

struct TokenizerModel {
    std::int32_t bos_id = -1;
    std::int32_t eos_id = -1;
    std::int32_t unk_id = -1;
    std::string chat_template_hash;
    std::vector<TokenEntry> vocab;
};

[[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);

[[nodiscard]] std::vector<std::int32_t> encode_prompt(
    const TokenizerModel& tokenizer,
    std::string_view prompt);

[[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer);

}  // namespace lcqi
```

三、核心实现

Listing 16.11 src/inference.cpp

```
#include <lcqi/inference.hpp>

#include <algorithm>
#include <cmath>
#include <stdexcept>

namespace lcqi {
```

```cpp
namespace {

void validate_input_size(std::span<const float> input, std::int32_t expected) {
    if (input.size() != static_cast<std::size_t>(expected)) {
        throw std::runtime_error("input size does not match model shape");
    }
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty logits");
    }
    std::int32_t best_index = 0;
    float best_value = values[0];
    for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i) {
        const float candidate = values[static_cast<std::size_t>(i)];
        if (candidate > best_value) {
            best_value = candidate;
            best_index = i;
        }
    }
    return best_index;
}

}  // namespace

void linear_i8(const QuantizedLinearLayer& layer,
               std::span<const float> input,
               std::span<float> output) {
    if (input.size() != static_cast<std::size_t>(layer.input_size)) {
        throw std::runtime_error("linear_i8 input size mismatch");
    }
    if (output.size() != static_cast<std::size_t>(layer.output_size)) {
        throw std::runtime_error("linear_i8 output size mismatch");
    }

    for (std::int32_t out = 0; out < layer.output_size; ++out) {
        const std::size_t row_base =
            static_cast<std::size_t>(out) * static_cast<std::size_t>(layer.input_size);
        float acc = layer.bias[static_cast<std::size_t>(out)];
        for (std::int32_t in = 0; in < layer.input_size; ++in) {
            const std::int8_t weight =
                layer.weights[row_base + static_cast<std::size_t>(in)];
            acc += static_cast<float>(weight) * layer.scale *
                   input[static_cast<std::size_t>(in)];
        }
        output[static_cast<std::size_t>(out)] = acc;
    }
}

```

```cpp
void relu_inplace(std::span<float> values) {
    for (float& value : values) {
        value = std::max(0.0F, value);
    }
}

InferenceResult run_inference(const TinyMlpModel& model,
                              std::span<const float> input) {
    validate_input_size(input, model.input_size);

    std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F);
    std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F);

    linear_i8(model.hidden, input, hidden);
    relu_inplace(hidden);
    linear_i8(model.output, hidden, logits);

    InferenceResult result;
    result.predicted_class = argmax(logits);
    result.logits = std::move(logits);
    return result;
}

}  // namespace lcqi
```

Listing 16.12 src/int8_kernels.cpp

```cpp
#include <lcqi/int8_kernels.hpp>

#include <lcqi/inference.hpp>

#include <algorithm>
#include <chrono>
#include <cmath>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;
constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
```

```cpp
constexpr const char* LCQI_BACKEND_AVX2 = "avx2";

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

void validate_quantized_layer(const QuantizedLinearLayer& layer) {
    require_positive(layer.input_size, "input_size");
    require_positive(layer.output_size, "output_size");
    const std::size_t expected_weights =
        checked_size(layer.input_size, "input_size") *
        checked_size(layer.output_size, "output_size");
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error("quantized layer weight size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("quantized layer bias size mismatch");
    }
}

std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
    require_positive(value, "value");
    require_positive(block_size, "block_size");
    return ((value + block_size - 1) / block_size) * block_size;
}

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
    result.backends.push_back(KernelBackendBenchmark{
        .name = name,
        .available = false,
        .average_us = 0.0,
        .max_abs_diff = 0.0F,
        .checksum = 0.0F,
    });
}
```

```
76
77 template <typename Callable>
78 double measure_average_us(std::int32_t repeat, Callable&& callable) {
79     require_positive(repeat, "repeat");
80     const auto begin = std::chrono::steady_clock::now();
81     for (std::int32_t index = 0; index < repeat; ++index) {
82         callable();
83     }
84     const auto end = std::chrono::steady_clock::now();
85     const std::chrono::duration<double> elapsed = end - begin;
86     return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
87 }
88
89 }  // namespace
90
91 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
92                               std::int32_t output_block_size) {
93     validate_quantized_layer(layer);
94     require_positive(output_block_size, "output_block_size");
95
96     PackedLinearI8 packed;
97     packed.input_size = layer.input_size;
98     packed.output_size = layer.output_size;
99     packed.output_block_size = output_block_size;
100    packed.scale = layer.scale;
101    packed.bias = layer.bias;
102
103    const std::int32_t padded_outputs =
104        round_up_to_block(layer.output_size, output_block_size);
105    packed.packed_weights.assign(
106        checked_size(padded_outputs, "padded_outputs") *
107            checked_size(layer.input_size, "input_size"),
108        0);
109
110    std::size_t packed_index = 0;
111    for (std::int32_t output_block = 0; output_block < padded_outputs;
112         output_block += output_block_size) {
113        for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
114            for (std::int32_t offset = 0; offset < output_block_size; ++offset) {
115                const std::int32_t output_index = output_block + offset;
116                if (output_index < layer.output_size) {
117                    const std::size_t source =
118                        checked_size(output_index, "output_index") *
119                            checked_size(layer.input_size, "input_size") +
120                        checked_size(input_index, "input_index");
121                    packed.packed_weights[packed_index] = layer.weights[source];
122                }
123                ++packed_index;
```

```
            }
        }
    }
    return packed;
}

void linear_i8_packed(const PackedLinearI8& layer,
                      std::span<const float> input,
                      std::span<float> output) {
    std::vector<float> accumulators(
        checked_size(layer.output_block_size, "output_block_size"),
        0.0F);
    linear_i8_packed_with_workspace(layer, input, output, accumulators);
}

void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
                                     std::span<const float> input,
                                     std::span<float> output,
                                     std::span<float> accumulator_workspace) {
    require_positive(layer.input_size, "packed input_size");
    require_positive(layer.output_size, "packed output_size");
    require_positive(layer.output_block_size, "packed output_block_size");
    if (input.size() != checked_size(layer.input_size, "input_size")) {
        throw std::runtime_error("linear_i8_packed input size mismatch");
    }
    if (output.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("linear_i8_packed output size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("linear_i8_packed bias size mismatch");
    }

    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    const std::size_t expected_packed_size =
        checked_size(padded_outputs, "padded_outputs") *
        checked_size(layer.input_size, "input_size");
    if (layer.packed_weights.size() != expected_packed_size) {
        throw std::runtime_error("linear_i8_packed packed weight size mismatch"
    ↪ );
    }
    if (accumulator_workspace.size() != checked_size(layer.output_block_size, "
    ↪ output_block_size")) {
        throw std::runtime_error("linear_i8_packed accumulator workspace size 
    ↪ mismatch");
    }

    std::size_t packed_index = 0;
    for (std::int32_t output_block = 0; output_block < padded_outputs;
         output_block += layer.output_block_size) {
        std::fill(accumulator_workspace.begin(), accumulator_workspace.end(), 
    ↪ 0.0F);
```

```
        for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                accumulator_workspace[checked_size(offset, "offset")] =
                    layer.bias[checked_size(output_index, "output_index")];
            }
        }

        for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
            const float input_value = input[checked_size(input_index, "input_index")];
            for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
                const std::int32_t output_index = output_block + offset;
                const std::int8_t weight = layer.packed_weights[packed_index++];
                if (output_index < layer.output_size) {
                    accumulator_workspace[checked_size(offset, "offset")] +=
                        static_cast<float>(weight) * layer.scale * input_value;
                }
            }
        }

        for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                output[checked_size(output_index, "output_index")] =
                    accumulator_workspace[checked_size(offset, "offset")];
            }
        }
    }
}

QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
                                                 std::int32_t output_size,
                                                 float scale) {
    require_positive(input_size, "input_size");
    require_positive(output_size, "output_size");
    QuantizedLinearLayer layer;
    layer.input_size = input_size;
    layer.output_size = output_size;
    layer.scale = scale;
    const std::size_t weight_count =
        checked_size(input_size, "input_size") * checked_size(output_size, "output_size");
    layer.weights.resize(weight_count);
    layer.bias.resize(checked_size(output_size, "output_size"));
    for (std::int32_t output_index = 0; output_index < output_size; ++output_index) {
```

```
        layer.bias[checked_size(output_index, "output_index")] =
            static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
                               LCQI_INPUT_PATTERN_CENTER) *
            LCQI_INPUT_PATTERN_SCALE;
        for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
            const std::int32_t raw =
                ((output_index * input_size + input_index) % LCQI_I8_PATTERN_MODULUS) -
                LCQI_I8_PATTERN_CENTER;
            layer.weights[checked_size(output_index, "output_index") *
                              checked_size(input_size, "input_size") +
                          checked_size(input_index, "input_index")] =
                static_cast<std::int8_t>(raw);
        }
    }
    return layer;
}

std::vector<float> make_deterministic_input(std::int32_t input_size) {
    require_positive(input_size, "input_size");
    std::vector<float> input(checked_size(input_size, "input_size"));
    for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
        input[checked_size(input_index, "input_index")] =
            static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
                               LCQI_INPUT_PATTERN_CENTER) *
            LCQI_INPUT_PATTERN_SCALE;
    }
    return input;
}

float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
    if (lhs.size() != rhs.size()) {
        throw std::runtime_error("max_abs_diff size mismatch");
    }
    float diff = 0.0F;
    for (std::size_t index = 0; index < lhs.size(); ++index) {
        diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
    }
    return diff;
}

KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& benchmark_case) {
    require_positive(benchmark_case.input_size, "benchmark input_size");
    require_positive(benchmark_case.output_size, "benchmark output_size");
    require_positive(benchmark_case.output_block_size, "benchmark output_block_size");
    require_positive(benchmark_case.repeat, "benchmark repeat");

    const QuantizedLinearLayer layer =
```

```
263         make_deterministic_i8_layer(benchmark_case.input_size,
264                                     benchmark_case.output_size,
265                                     0.015625F);
266     const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.↩
    ↪ output_block_size);
267     const PackedLinearI8 packed_simd = pack_linear_i8(layer, ↩
    ↪ LCQI_SIMD_OUTPUT_BLOCK);
268     const std::vector<float> input = make_deterministic_input(benchmark_case.↩
    ↪ input_size);
269     std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "↩
    ↪ output_size"),
270                                      0.0F);
271     std::vector<float> packed_output(scalar_output.size(), 0.0F);
272     std::vector<float> simd_output(scalar_output.size(), 0.0F);
273     std::vector<float> packed_workspace(
274         checked_size(benchmark_case.output_block_size, "output_block_size"),
275         0.0F);
276 
277     linear_i8(layer, input, scalar_output);
278     linear_i8_packed_with_workspace(packed, input, packed_output, ↩
    ↪ packed_workspace);
279 
280     KernelBenchmarkResult result;
281     result.benchmark_case = benchmark_case;
282     result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
283     result.backends.push_back(KernelBackendBenchmark{
284         .name = LCQI_BACKEND_SCALAR,
285         .available = true,
286         .average_us = measure_average_us(benchmark_case.repeat,
287                                          [&]() { linear_i8(layer, input, ↩
    ↪ scalar_output); }),
288         .max_abs_diff = 0.0F,
289         .checksum = checksum(scalar_output),
290     });
291     result.backends.push_back(KernelBackendBenchmark{
292         .name = LCQI_BACKEND_PACKED_SCALAR,
293         .available = true,
294         .average_us = measure_average_us(
295             benchmark_case.repeat,
296             [&]() { linear_i8_packed_with_workspace(packed,
297                                                     input,
298                                                     packed_output,
299                                                     packed_workspace); }),
300         .max_abs_diff = max_abs_diff(scalar_output, packed_output),
301         .checksum = checksum(packed_output),
302     });
303     if (linear_i8_packed_avx2_available()) {
304         linear_i8_packed_avx2(packed_simd, input, simd_output);
305         result.backends.push_back(KernelBackendBenchmark{
306             .name = LCQI_BACKEND_AVX2,
307             .available = true,
308             .average_us = measure_average_us(
```

```
                benchmark_case.repeat,
                [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output); }),
            .max_abs_diff = max_abs_diff(scalar_output, simd_output),
            .checksum = checksum(simd_output),
        });
    } else {
        add_unavailable_backend(result, LCQI_BACKEND_AVX2);
    }
    return result;
}

}  // namespace lcqi

#if !defined(LCQI_ENABLE_AVX2)
namespace lcqi {

bool linear_i8_packed_avx2_available() noexcept {
    return false;
}

void linear_i8_packed_avx2(const PackedLinearI8&, std::span<const float>, std::span<float>) {
    throw std::runtime_error("AVX2 packed kernel is not enabled in this build");
}

}  // namespace lcqi
#endif
```

Listing 16.13 src/int8_kernels_avx2.cpp

```
#include <lcqi/int8_kernels.hpp>

#if defined(LCQI_ENABLE_AVX2)

#include <immintrin.h>

#include <algorithm>
#include <cstdint>
#include <cstring>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_AVX2_OUTPUT_BLOCK = 8;

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
```

```
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
    require_positive(value, "value");
    require_positive(block_size, "block_size");
    return ((value + block_size - 1) / block_size) * block_size;
}

void validate_avx2_contract(const PackedLinearI8& layer,
                            std::span<const float> input,
                            std::span<float> output) {
    require_positive(layer.input_size, "packed input_size");
    require_positive(layer.output_size, "packed output_size");
    if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
        throw std::runtime_error("AVX2 packed kernel requires output_block_size
    ↪  == 8");
    }
    if (input.size() != checked_size(layer.input_size, "input_size")) {
        throw std::runtime_error("AVX2 packed kernel input size mismatch");
    }
    if (output.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("AVX2 packed kernel output size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("AVX2 packed kernel bias size mismatch");
    }
    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    const std::size_t expected_packed_size =
        checked_size(padded_outputs, "padded_outputs") *
        checked_size(layer.input_size, "input_size");
    if (layer.packed_weights.size() != expected_packed_size) {
        throw std::runtime_error("AVX2 packed kernel packed weight size
    ↪ mismatch");
    }
}

}  // namespace

bool linear_i8_packed_avx2_available() noexcept {
#if defined(__GNUC__) || defined(__clang__)
    __builtin_cpu_init();
    return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
#else
```

```
    return true;
#endif
}

void linear_i8_packed_avx2(const PackedLinearI8& layer,
                           std::span<const float> input,
                           std::span<float> output) {
    if (!linear_i8_packed_avx2_available()) {
        throw std::runtime_error("AVX2/FMA is not available on this CPU");
    }
    validate_avx2_contract(layer, input, output);

    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    std::size_t packed_index = 0;
    alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};

    for (std::int32_t output_block = 0; output_block < padded_outputs;
         output_block += LCQI_AVX2_OUTPUT_BLOCK) {
        __m256 accumulator = _mm256_setzero_ps();

        for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset
    ↪ ) {
            const std::int32_t output_index = output_block + offset;
            block_output[checked_size(offset, "offset")] =
                output_index < layer.output_size
                    ? layer.bias[checked_size(output_index, "output_index")]
                    : 0.0F;
        }
        accumulator = _mm256_load_ps(block_output);

        for (std::int32_t input_index = 0; input_index < layer.input_size; ++
    ↪ input_index) {
            std::uint64_t packed_bytes = 0;
            std::memcpy(&packed_bytes,
                        layer.packed_weights.data() + packed_index,
                        sizeof(packed_bytes));
            const __m128i packed_i8 =
                _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
            packed_index += LCQI_AVX2_OUTPUT_BLOCK;
            const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
            const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
            const __m256 input_scaled =
                _mm256_set1_ps(input[checked_size(input_index, "input_index")] ↩
    ↪ * layer.scale);
            accumulator = _mm256_fmadd_ps(weights_f32, input_scaled, ↩
    ↪ accumulator);
        }

        _mm256_store_ps(block_output, accumulator);
        for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset
    ↪ ) {
```

```
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                output[checked_size(output_index, "output_index")] =
                    block_output[checked_size(offset, "offset")];
            }
        }
    }
}

}  // namespace lcqi

#endif
```

Listing 16.14 src/model.cpp

```
#include <lcqi/model.hpp>

#include <fstream>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;

std::string read_key(std::istream& input) {
    std::string key;
    if (!(input >> key)) {
        throw std::runtime_error("unexpected end of model file");
    }
    return key;
}

void expect_key(std::istream& input, const std::string& expected) {
    const std::string key = read_key(input);
    if (key != expected) {
        throw std::runtime_error("expected key '" + expected + "', got '" + key
    ↪  + "'");
    }
}

std::int32_t read_i32(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::int32_t value = 0;
    if (!(input >> value)) {
        throw std::runtime_error("invalid int32 value for key '" + key + "'");
    }
    return value;
}
```

```
37
38 float read_float(std::istream& input, const std::string& key) {
39     expect_key(input, key);
40     float value = 0.0F;
41     if (!(input >> value)) {
42         throw std::runtime_error("invalid float value for key '" + key + "'");
43     }
44     return value;
45 }
46
47 std::vector<std::int8_t> read_i8_vector(std::istream& input,
48                                         const std::string& key,
49                                         std::int32_t count) {
50     expect_key(input, key);
51     if (count < 0) {
52         throw std::runtime_error("negative vector size for key '" + key + "'");
53     }
54     std::vector<std::int8_t> values(static_cast<std::size_t>(count));
55     for (std::int32_t i = 0; i < count; ++i) {
56         std::int32_t raw = 0;
57         if (!(input >> raw)) {
58             throw std::runtime_error("invalid int8 payload for key '" + key + "↩
    ↪ '");
59         }
60         if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
61             throw std::runtime_error("int8 payload out of range for key '" + ↩
    ↪ key + "'");
62         }
63         values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
64     }
65     return values;
66 }
67
68 std::vector<float> read_float_vector(std::istream& input,
69                                      const std::string& key,
70                                      std::int32_t count) {
71     expect_key(input, key);
72     if (count < 0) {
73         throw std::runtime_error("negative vector size for key '" + key + "'");
74     }
75     std::vector<float> values(static_cast<std::size_t>(count));
76     for (std::int32_t i = 0; i < count; ++i) {
77         if (!(input >> values[static_cast<std::size_t>(i)])) {
78             throw std::runtime_error("invalid float payload for key '" + key + ↩
    ↪ "'");
79         }
80     }
81     return values;
82 }
83
84 void validate_layer(const QuantizedLinearLayer& layer, const std::string& name)↩
    ↪  {
```

```
    if (layer.input_size <= 0 || layer.output_size <= 0) {
        throw std::runtime_error(name + " layer has invalid shape");
    }
    const std::size_t expected_weights =
        static_cast<std::size_t>(layer.input_size) *
        static_cast<std::size_t>(layer.output_size);
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error(name + " layer weight size mismatch");
    }
    if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
        throw std::runtime_error(name + " layer bias size mismatch");
    }
}

std::int32_t checked_element_count(std::int32_t rows,
                                   std::int32_t columns,
                                   const std::string& name) {
    if (rows <= 0 || columns <= 0) {
        throw std::runtime_error(name + " layer has invalid shape");
    }
    const std::int64_t count =
        static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
    if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::
    ↪ max())) {
        throw std::runtime_error(name + " layer element count is too large");
    }
    return static_cast<std::int32_t>(count);
}

}  // namespace

TinyMlpModel load_model(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open model file: " + path.string());
    }

    expect_key(input, "LCQI_MODEL_V1");

    TinyMlpModel model;
    model.input_size = read_i32(input, "input_size");
    model.hidden_size = read_i32(input, "hidden_size");
    model.output_size = read_i32(input, "output_size");

    const std::int32_t hidden_weight_count =
        checked_element_count(model.input_size, model.hidden_size, "hidden");
    const std::int32_t output_weight_count =
        checked_element_count(model.hidden_size, model.output_size, "output");

    model.hidden.input_size = model.input_size;
    model.hidden.output_size = model.hidden_size;
    model.hidden.scale = read_float(input, "hidden_scale");
```

```
    model.hidden.weights = read_i8_vector(
        input,
        "hidden_weights",
        hidden_weight_count);
    model.hidden.bias = read_float_vector(input, "hidden_bias", model.hidden_size);

    model.output.input_size = model.hidden_size;
    model.output.output_size = model.output_size;
    model.output.scale = read_float(input, "output_scale");
    model.output.weights = read_i8_vector(
        input,
        "output_weights",
        output_weight_count);
    model.output.bias = read_float_vector(input, "output_bias", model.output_size);

    validate_layer(model.hidden, "hidden");
    validate_layer(model.output, "output");
    return model;
}

}  // namespace lcqi
```

Listing 16.15 src/reference_decoder.cpp

```
#include <lcqi/reference_decoder.hpp>

#include <algorithm>
#include <array>
#include <cmath>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::infinity();
constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

```

```cpp
void validate_config(const DecoderConfig& config) {
    require_positive(config.hidden_size, "hidden_size");
    require_positive(config.query_heads, "query_heads");
    require_positive(config.kv_heads, "kv_heads");
    require_positive(config.head_dim, "head_dim");
    require_positive(config.intermediate_size, "intermediate_size");
    require_positive(config.vocab_size, "vocab_size");
    require_positive(config.max_context, "max_context");
    if (config.query_heads % config.kv_heads != 0) {
        throw std::runtime_error("query_heads must be divisible by kv_heads");
    }
    if (config.hidden_size != config.query_heads * config.head_dim) {
        throw std::runtime_error("hidden_size must equal query_heads * head_dim");
    }
    if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
        throw std::runtime_error("head_dim must be even for RoPE");
    }
    if (config.rope_base <= 1.0F) {
        throw std::runtime_error("rope_base must be greater than 1");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

std::int32_t kv_width(const DecoderConfig& config) {
    return config.kv_heads * config.head_dim;
}

void validate_linear(const LinearWeightsF32& layer, const char* name) {
    require_positive(layer.input_size, "linear input_size");
    require_positive(layer.output_size, "linear output_size");
    const std::size_t expected_weights =
        checked_size(layer.input_size, name) * checked_size(layer.output_size, name);
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error(std::string(name) + " weight size mismatch");
    }
    if (!layer.bias.empty() &&
        layer.bias.size() != checked_size(layer.output_size, name)) {
        throw std::runtime_error(std::string(name) + " bias size mismatch");
    }
}

void validate_span_size(std::size_t actual, std::int32_t expected, const char* name) {
    if (actual != checked_size(expected, name)) {
```

```
        throw std::runtime_error(std::string(name) + " size mismatch");
    }
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty values");
    }
    std::int32_t best_index = 0;
    float best_value = values[0];
    for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()); ++index) {
        const float candidate = values[static_cast<std::size_t>(index)];
        if (candidate > best_value) {
            best_value = candidate;
            best_index = index;
        }
    }
    return best_index;
}

float silu(float value) {
    return value / (1.0F + std::exp(-value));
}

std::span<const float> row_span(std::span<const float> values,
                                std::int32_t row,
                                std::int32_t row_width) {
    const std::size_t begin = checked_size(row, "row") * checked_size(row_width, "row_width");
    return values.subspan(begin, checked_size(row_width, "row_width"));
}

void add_inplace(std::span<float> target, std::span<const float> source) {
    if (target.size() != source.size()) {
        throw std::runtime_error("residual add size mismatch");
    }
    for (std::size_t index = 0; index < target.size(); ++index) {
        target[index] += source[index];
    }
}

void swiglu_mlp(const DecoderLayerWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output) {
    std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F);
    std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
    std::vector<float> hidden(gate.size(), 0.0F);
    linear_f32(layer.w_gate, input, gate);
    linear_f32(layer.w_up, input, up);
    if (gate.size() != up.size()) {
```

```
        throw std::runtime_error("SwiGLU gate/up size mismatch");
    }
    for (std::size_t index = 0; index < hidden.size(); ++index) {
        hidden[index] = silu(gate[index]) * up[index];
    }
    linear_f32(layer.w_down, hidden, output);
}

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

float max_abs(std::span<const float> values) {
    float result = 0.0F;
    for (const float value : values) {
        result = std::max(result, std::fabs(value));
    }
    return result;
}

std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape
    ) {
    std::vector<std::int32_t> stride(shape.size(), 1);
    std::int32_t running = 1;
    for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; 
    index >= 0; --index) {
        stride[checked_size(index, "stride index")] = running;
        running *= shape[checked_size(index, "shape index")];
    }
    return stride;
}

void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
                    const char* name,
                    std::int32_t token_position,
                    std::int32_t layer_id,
                    std::span<const std::int32_t> shape,
                    const char* layout,
                    std::span<const float> values) {
    if (checkpoints == nullptr) {
        return;
    }
    ReferenceTensorCheckpoint checkpoint;
    checkpoint.name = name;
    checkpoint.token_position = token_position;
    checkpoint.layer_id = layer_id;
    checkpoint.shape.assign(shape.begin(), shape.end());
    checkpoint.stride = contiguous_stride(shape);
```

```
    checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
    checkpoint.layout = layout;
    checkpoint.checksum = checksum(values);
    checkpoint.max_abs = max_abs(values);
    checkpoint.values.assign(values.begin(), values.end());
    checkpoints->push_back(std::move(checkpoint));
}

void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
                           std::span<const float> input,
                           std::span<float> output,
                           std::int32_t token_position,
                           std::int32_t layer_id,
                           std::vector<ReferenceTensorCheckpoint>* checkpoints) {
    std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F);
    std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
    std::vector<float> hidden(gate.size(), 0.0F);
    linear_f32(layer.w_gate, input, gate);
    linear_f32(layer.w_up, input, up);
    if (gate.size() != up.size()) {
        throw std::runtime_error("SwiGLU gate/up size mismatch");
    }
    const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.output_size};
    add_checkpoint(checkpoints,
                   "mlp_gate",
                   token_position,
                   layer_id,
                   intermediate_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   gate);
    add_checkpoint(checkpoints,
                   "mlp_up",
                   token_position,
                   layer_id,
                   intermediate_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   up);
    for (std::size_t index = 0; index < hidden.size(); ++index) {
        hidden[index] = silu(gate[index]) * up[index];
    }
    add_checkpoint(checkpoints,
                   "mlp_hidden",
                   token_position,
                   layer_id,
                   intermediate_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   hidden);
    linear_f32(layer.w_down, hidden, output);
}
```

```
223
224 void forward_layer(const DecoderConfig& config,
225                     const DecoderLayerWeightsF32& layer,
226                     std::int32_t layer_id,
227                     std::int32_t model_position,
228                     ReferenceKVCache& cache,
229                     std::span<float> hidden,
230                     std::vector<ReferenceTensorCheckpoint>* checkpoints =
    ↪ nullptr) {
231     validate_span_size(hidden.size(), config.hidden_size, "hidden");
232
233     std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"),
    ↪ 0.0F);
234     std::vector<float> query(checked_size(config.hidden_size, "hidden_size"),
    ↪ 0.0F);
235     std::vector<float> key(checked_size(kv_width(config), "kv_width"), 0.0F);
236     std::vector<float> value(checked_size(kv_width(config), "kv_width"), 0.0F);
237     std::vector<float> attention(checked_size(config.hidden_size, "hidden_size"
    ↪ ), 0.0F);
238     std::vector<float> attention_projected(checked_size(config.hidden_size, "
    ↪ hidden_size"), 0.0F);
239     std::vector<float> mlp(checked_size(config.hidden_size, "hidden_size"), 0.0
    ↪ F);
240
241     const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
242     const std::array<std::int32_t, 2> query_shape{config.query_heads, config.
    ↪ head_dim};
243     const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim
    ↪ };
244     const std::array<std::int32_t, 4> kv_slot_shape{
245         1, 1, config.kv_heads, config.head_dim};
246
247     rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
248     add_checkpoint(checkpoints,
249                    "attn_norm",
250                    model_position,
251                    layer_id,
252                    hidden_shape,
253                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
254                    normed);
255     linear_f32(layer.wq, normed, query);
256     linear_f32(layer.wk, normed, key);
257     linear_f32(layer.wv, normed, value);
258     add_checkpoint(checkpoints,
259                    "q_before_rope",
260                    model_position,
261                    layer_id,
262                    query_shape,
263                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
264                    query);
265     add_checkpoint(checkpoints,
266                    "k_before_rope",
```

```
                    model_position,
                    layer_id,
                    kv_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    key);
    add_checkpoint(checkpoints,
                    "v",
                    model_position,
                    layer_id,
                    kv_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    value);
    apply_rope(query, config.query_heads, config.head_dim, model_position, config.rope_base);
    apply_rope(key, config.kv_heads, config.head_dim, model_position, config.rope_base);
    add_checkpoint(checkpoints,
                    "q_after_rope",
                    model_position,
                    layer_id,
                    query_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    query);
    add_checkpoint(checkpoints,
                    "k_after_rope",
                    model_position,
                    layer_id,
                    kv_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    key);

    cache.append(layer_id, model_position, key, value);
    add_checkpoint(checkpoints,
                    "kv_cache_key_slot",
                    model_position,
                    layer_id,
                    kv_slot_shape,
                    LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
                    key);
    attention_decode(config, cache, layer_id, model_position, query, attention);
    add_checkpoint(checkpoints,
                    "attention_out",
                    model_position,
                    layer_id,
                    hidden_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    attention);
    linear_f32(layer.wo, attention, attention_projected);
    add_inplace(hidden, attention_projected);
    add_checkpoint(checkpoints,
                    "post_attention_residual",
```

```
                    model_position,
                    layer_id,
                    hidden_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    hidden);

    rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);
    add_checkpoint(checkpoints,
                    "mlp_norm",
                    model_position,
                    layer_id,
                    hidden_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    normed);
    if (checkpoints == nullptr) {
        swiglu_mlp(layer, normed, mlp);
    } else {
        swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, ↩
    ↪ checkpoints);
    }
    add_inplace(hidden, mlp);
    add_checkpoint(checkpoints,
                    "layer_out",
                    model_position,
                    layer_id,
                    hidden_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    hidden);
}

LinearWeightsF32 make_linear(std::int32_t input_size,
                             std::int32_t output_size,
                             std::vector<float> weights,
                             std::vector<float> bias = {}) {
    LinearWeightsF32 layer;
    layer.input_size = input_size;
    layer.output_size = output_size;
    layer.weights = std::move(weights);
    layer.bias = std::move(bias);
    validate_linear(layer, "tiny linear");
    return layer;
}

std::vector<float> zeros(std::int32_t count) {
    return std::vector<float>(checked_size(count, "zero count"), 0.0F);
}

}  // namespace

ReferenceDecodeTraceResult run_reference_decode_with_trace(
    const ReferenceDecoderModel& model,
    std::span<const std::int32_t> token_ids) {
```

```
367     validate_config(model.config);
368     if (token_ids.empty()) {
369         throw std::runtime_error("reference decode needs at least one token");
370     }
371     if (token_ids.size() > checked_size(model.config.max_context, "max_context"
    ↪ )) {
372         throw std::runtime_error("token_ids exceed max_context");
373     }
374     if (model.token_embedding.size() !=
375         checked_size(model.config.vocab_size, "vocab_size") *
376             checked_size(model.config.hidden_size, "hidden_size")) {
377         throw std::runtime_error("token embedding size mismatch");
378     }
379     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size
    ↪ , "hidden_size")) {
380         throw std::runtime_error("final norm weight size mismatch");
381     }
382     validate_linear(model.lm_head, "lm_head");
383 
384     ReferenceDecodeTraceResult trace;
385     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers
    ↪ .size()));
386     std::vector<float> hidden(checked_size(model.config.hidden_size, "
    ↪ hidden_size"), 0.0F);
387     const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
388     for (std::int32_t position = 0; position < static_cast<std::int32_t>(
    ↪ token_ids.size());
389          ++position) {
390         const std::int32_t token_id = token_ids[checked_size(position, "
    ↪ position")];
391         if (token_id < 0 || token_id >= model.config.vocab_size) {
392             throw std::runtime_error("token id out of vocabulary");
393         }
394         const std::span<const float> embedding =
395             row_span(model.token_embedding, token_id, model.config.hidden_size)
    ↪ ;
396         std::copy(embedding.begin(), embedding.end(), hidden.begin());
397         add_checkpoint(&trace.checkpoints,
398                        "embedding",
399                        position,
400                        -1,
401                        hidden_shape,
402                        LCQI_TRACE_LAYOUT_ROW_MAJOR,
403                        hidden);
404         for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(
    ↪ model.layers.size());
405              ++layer_id) {
406             forward_layer(model.config,
407                           model.layers[checked_size(layer_id, "layer_id")],
408                           layer_id,
409                           position,
410                           cache,
```

```
                            hidden,
                            &trace.checkpoints);
        }
    }

    std::vector<float> normed(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size"), 0.0F);
    const std::array<std::int32_t, 1> logits_shape{model.config.vocab_size};
    const std::int32_t last_position = static_cast<std::int32_t>(token_ids.size()) - 1;
    rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed);
    add_checkpoint(&trace.checkpoints,
                   "final_norm",
                   last_position,
                   -1,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   normed);
    linear_f32(model.lm_head, normed, logits);
    add_checkpoint(&trace.checkpoints,
                   "logits",
                   last_position,
                   -1,
                   logits_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   logits);

    trace.result.predicted_token = argmax(logits);
    trace.result.logits = std::move(logits);
    return trace;
}

ReferenceKVCache::ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count)
    : config_(config),
      layer_count_(layer_count) {
    validate_config(this->config_);
    require_positive(this->layer_count_, "layer_count");
    const std::size_t element_count =
        checked_size(this->layer_count_, "layer_count") *
        checked_size(this->config_.max_context, "max_context") *
        checked_size(this->config_.kv_heads, "kv_heads") *
        checked_size(this->config_.head_dim, "head_dim");
    this->keys_.assign(element_count, 0.0F);
    this->values_.assign(element_count, 0.0F);
    this->written_.assign(
        checked_size(this->layer_count_, "layer_count") *
            checked_size(this->config_.max_context, "max_context"),
        0);
```

```
}

void ReferenceKVCache::append(std::int32_t layer_id,
                              std::int32_t model_position,
                              std::span<const float> key,
                              std::span<const float> value) {
    validate_span_size(key.size(), kv_width(this->config_), "KV key");
    validate_span_size(value.size(), kv_width(this->config_), "KV value");
    this->validate_address(layer_id, model_position, 0);
    for (std::int32_t kv_head = 0; kv_head < this->config_.kv_heads; ++kv_head) {
        const std::size_t target = this->base_offset(layer_id, model_position, kv_head);
        const std::size_t source =
            checked_size(kv_head, "kv_head") * checked_size(this->config_.head_dim, "head_dim");
        std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
                    this->config_.head_dim,
                    this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
        std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
                    this->config_.head_dim,
                    this->values_.begin() + static_cast<std::ptrdiff_t>(target));
    }
    const std::size_t written_index =
        checked_size(layer_id, "layer_id") * checked_size(this->config_.max_context, "max_context") +
        checked_size(model_position, "model_position");
    this->written_[written_index] = 1;
    this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
}

std::span<const float> ReferenceKVCache::key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t kv_head) const {
    this->validate_address(layer_id, model_position, kv_head);
    const std::size_t offset = this->base_offset(layer_id, model_position, kv_head);
    return std::span<const float>(this->keys_.data() + offset,
                                  checked_size(this->config_.head_dim, "head_dim"));
}

std::span<const float> ReferenceKVCache::value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t kv_head) const {
    this->validate_address(layer_id, model_position, kv_head);
    const std::size_t offset = this->base_offset(layer_id, model_position, kv_head);
    return std::span<const float>(this->values_.data() + offset,
                                  checked_size(this->config_.head_dim, "head_dim"));
```

```cpp
}

std::int32_t ReferenceKVCache::layer_count() const noexcept {
    return this->layer_count_;
}

std::int32_t ReferenceKVCache::filled_tokens() const noexcept {
    return this->filled_tokens_;
}

std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t kv_head) const {
    return (((checked_size(layer_id, "layer_id") *
              checked_size(this->config_.max_context, "max_context")) +
             checked_size(model_position, "model_position")) *
                checked_size(this->config_.kv_heads, "kv_heads") +
            checked_size(kv_head, "kv_head")) *
           checked_size(this->config_.head_dim, "head_dim");
}

void ReferenceKVCache::validate_address(std::int32_t layer_id,
                                        std::int32_t model_position,
                                        std::int32_t kv_head) const {
    if (layer_id < 0 || layer_id >= this->layer_count_) {
        throw std::runtime_error("KV layer_id out of range");
    }
    if (model_position < 0 || model_position >= this->config_.max_context) {
        throw std::runtime_error("KV model_position out of range");
    }
    if (kv_head < 0 || kv_head >= this->config_.kv_heads) {
        throw std::runtime_error("KV head out of range");
    }
}

void rms_norm(std::span<const float> input,
              std::span<const float> weight,
              float epsilon,
              std::span<float> output) {
    if (input.size() != weight.size() || input.size() != output.size()) {
        throw std::runtime_error("RMSNorm size mismatch");
    }
    if (input.empty()) {
        throw std::runtime_error("RMSNorm input is empty");
    }
    float sum_square = 0.0F;
    for (const float value : input) {
        sum_square += value * value;
    }
    const float mean_square = sum_square / static_cast<float>(input.size());
    const float scale = 1.0F / std::sqrt(mean_square + epsilon);
    for (std::size_t index = 0; index < input.size(); ++index) {
```

```
553         output[index] = input[index] * scale * weight[index];
554     }
555 }
556 
557 void linear_f32(const LinearWeightsF32& layer,
558                 std::span<const float> input,
559                 std::span<float> output) {
560     validate_linear(layer, "linear_f32");
561     validate_span_size(input.size(), layer.input_size, "linear input");
562     validate_span_size(output.size(), layer.output_size, "linear output");
563     for (std::int32_t out = 0; out < layer.output_size; ++out) {
564         const std::size_t row_base =
565             checked_size(out, "out") * checked_size(layer.input_size, "↩
    ↪ input_size");
566         float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size↩
    ↪ (out, "out")];
567         for (std::int32_t in = 0; in < layer.input_size; ++in) {
568             accumulator += layer.weights[row_base + checked_size(in, "in")] *
569                            input[checked_size(in, "in")];
570         }
571         output[checked_size(out, "out")] = accumulator;
572     }
573 }
574 
575 void apply_rope(std::span<float> heads,
576                 std::int32_t head_count,
577                 std::int32_t head_dim,
578                 std::int32_t model_position,
579                 float rope_base) {
580     require_positive(head_count, "head_count");
581     require_positive(head_dim, "head_dim");
582     validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
583     if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
584         throw std::runtime_error("RoPE head_dim must be even");
585     }
586     if (model_position < 0) {
587         throw std::runtime_error("RoPE model_position cannot be negative");
588     }
589     if (rope_base <= 1.0F) {
590         throw std::runtime_error("RoPE base must be greater than 1");
591     }
592     for (std::int32_t head = 0; head < head_count; ++head) {
593         for (std::int32_t offset = 0; offset < head_dim; offset += ↩
    ↪ LCQI_ROPE_PAIR_WIDTH) {
594             const float exponent = static_cast<float>(offset) / static_cast<↩
    ↪ float>(head_dim);
595             const float theta = 1.0F / std::pow(rope_base, exponent);
596             const float angle = static_cast<float>(model_position) * theta;
597             const float cosine = std::cos(angle);
598             const float sine = std::sin(angle);
599             const std::size_t first =
600                 checked_size(head, "head") * checked_size(head_dim, "head_dim")↩
```

```
    ↪  +
                checked_size(offset, "offset");
            const float x0 = heads[first];
            const float x1 = heads[first + 1];
            heads[first] = x0 * cosine - x1 * sine;
            heads[first + 1] = x0 * sine + x1 * cosine;
        }
    }
}

void attention_decode(const DecoderConfig& config,
                      const ReferenceKVCache& cache,
                      std::int32_t layer_id,
                      std::int32_t model_position,
                      std::span<const float> query,
                      std::span<float> output) {
    validate_config(config);
    validate_span_size(query.size(), config.hidden_size, "attention query");
    validate_span_size(output.size(), config.hidden_size, "attention output");
    if (model_position < 0 || model_position >= cache.filled_tokens()) {
        throw std::runtime_error("attention model_position has not been written↩
    ↪ ");
    }
    std::fill(output.begin(), output.end(), 0.0F);

    const std::int32_t group_size = config.query_heads / config.kv_heads;
    const float score_scale = 1.0F / std::sqrt(static_cast<float>(config.↩
    ↪ head_dim));
    std::vector<float> scores(checked_size(model_position + 1, "score count"),
                              LCQI_SOFTMAX_NEGATIVE_INFINITY);
    std::vector<float> probabilities(scores.size(), 0.0F);

    for (std::int32_t query_head = 0; query_head < config.query_heads; ++↩
    ↪ query_head) {
        const std::int32_t kv_head = query_head / group_size;
        const std::size_t query_base =
            checked_size(query_head, "query_head") * checked_size(config.↩
    ↪ head_dim, "head_dim");
        float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
        for (std::int32_t position = 0; position <= model_position; ++position)↩
    ↪  {
            const std::span<const float> key = cache.key(layer_id, position, ↩
    ↪ kv_head);
            float dot = 0.0F;
            for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
                dot += query[query_base + checked_size(dim, "dim")] *
                       key[checked_size(dim, "dim")];
            }
            const float score = dot * score_scale;
            scores[checked_size(position, "position")] = score;
            max_score = std::max(max_score, score);
        }
```

```
646
647         float probability_sum = 0.0F;
648         for (std::int32_t position = 0; position <= model_position; ++position)↩
    ↪  {
649             const std::size_t index = checked_size(position, "position");
650             const float unnormalized = std::exp(scores[index] - max_score);
651             probabilities[index] = unnormalized;
652             probability_sum += unnormalized;
653         }
654         if (probability_sum <= 0.0F) {
655             throw std::runtime_error("attention probability sum is invalid");
656         }
657         for (std::int32_t position = 0; position <= model_position; ++position)↩
    ↪  {
658             const float probability =
659                 probabilities[checked_size(position, "position")] / ↩
    ↪ probability_sum;
660             const std::span<const float> value = cache.value(layer_id, position↩
    ↪ , kv_head);
661             for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
662                 output[query_base + checked_size(dim, "dim")] +=
663                     probability * value[checked_size(dim, "dim")];
664             }
665         }
666     }
667 }
668
669 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
670                                            std::span<const std::int32_t> ↩
    ↪ token_ids) {
671     validate_config(model.config);
672     if (token_ids.empty()) {
673         throw std::runtime_error("reference decode needs at least one token");
674     }
675     if (token_ids.size() > checked_size(model.config.max_context, "max_context"↩
    ↪ )) {
676         throw std::runtime_error("token_ids exceed max_context");
677     }
678     if (model.token_embedding.size() !=
679         checked_size(model.config.vocab_size, "vocab_size") *
680             checked_size(model.config.hidden_size, "hidden_size")) {
681         throw std::runtime_error("token embedding size mismatch");
682     }
683     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size↩
    ↪ , "hidden_size")) {
684         throw std::runtime_error("final norm weight size mismatch");
685     }
686     validate_linear(model.lm_head, "lm_head");
687
688     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers↩
    ↪ .size()));
689     std::vector<float> hidden(checked_size(model.config.hidden_size, "↩
```

```
    ↪ hidden_size"), 0.0F);
690     for (std::int32_t position = 0; position < static_cast<std::int32_t>(↩
    ↪ token_ids.size());
691          ++position) {
692         const std::int32_t token_id = token_ids[checked_size(position, "↩
    ↪ position")];
693         if (token_id < 0 || token_id >= model.config.vocab_size) {
694             throw std::runtime_error("token id out of vocabulary");
695         }
696         const std::span<const float> embedding =
697             row_span(model.token_embedding, token_id, model.config.hidden_size)↩
    ↪ ;
698         std::copy(embedding.begin(), embedding.end(), hidden.begin());
699         for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(↩
    ↪ model.layers.size());
700              ++layer_id) {
701             forward_layer(model.config,
702                           model.layers[checked_size(layer_id, "layer_id")],
703                           layer_id,
704                           position,
705                           cache,
706                           hidden);
707         }
708     }
709
710     std::vector<float> normed(checked_size(model.config.hidden_size, "↩
    ↪ hidden_size"), 0.0F);
711     std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size↩
    ↪ "), 0.0F);
712     rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)↩
    ↪ ;
713     linear_f32(model.lm_head, normed, logits);
714
715     ReferenceDecodeResult result;
716     result.predicted_token = argmax(logits);
717     result.logits = std::move(logits);
718     return result;
719 }
720
721 ReferenceDecoderModel make_tiny_reference_decoder_model() {
722     ReferenceDecoderModel model;
723     model.config.hidden_size = 4;
724     model.config.query_heads = 2;
725     model.config.kv_heads = 1;
726     model.config.head_dim = 2;
727     model.config.intermediate_size = 6;
728     model.config.vocab_size = 5;
729     model.config.max_context = 8;
730     model.config.rms_epsilon = 1.0e-5F;
731     model.config.rope_base = 10000.0F;
732
733     model.token_embedding = {
```

```
        0.20F, -0.10F, 0.05F, 0.30F,
        -0.40F, 0.10F, 0.25F, -0.20F,
        0.15F, 0.35F, -0.30F, 0.05F,
        0.50F, -0.25F, 0.10F, -0.15F,
        -0.05F, 0.20F, 0.40F, -0.35F,
    };

    DecoderLayerWeightsF32 layer;
    layer.rms_attention_weight = {1.0F, 0.9F, 1.1F, 1.0F};
    layer.wq = make_linear(4,
                           4,
                           {
                               0.20F, -0.10F, 0.05F, 0.30F,
                               -0.15F, 0.25F, 0.10F, -0.05F,
                               0.05F, 0.10F, -0.20F, 0.15F,
                               0.30F, -0.05F, 0.20F, 0.10F,
                           });
    layer.wk = make_linear(4,
                           2,
                           {
                               0.10F, 0.20F, -0.10F, 0.05F,
                               -0.05F, 0.15F, 0.25F, -0.20F,
                           });
    layer.wv = make_linear(4,
                           2,
                           {
                               0.25F, -0.05F, 0.10F, 0.15F,
                               -0.10F, 0.30F, 0.05F, 0.20F,
                           });
    layer.wo = make_linear(4,
                           4,
                           {
                               0.20F, 0.10F, -0.05F, 0.15F,
                               -0.10F, 0.25F, 0.20F, -0.05F,
                               0.05F, -0.20F, 0.30F, 0.10F,
                               0.15F, 0.05F, -0.10F, 0.25F,
                           });
    layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
    layer.w_gate = make_linear(4,
                               6,
                               {
                                   0.20F, -0.10F, 0.05F, 0.10F,
                                   -0.05F, 0.15F, 0.20F, -0.10F,
                                   0.10F, 0.05F, -0.15F, 0.25F,
                                   -0.20F, 0.10F, 0.15F, 0.05F,
                                   0.05F, -0.25F, 0.10F, 0.20F,
                                   0.15F, 0.05F, 0.05F, -0.15F,
                               });
    layer.w_up = make_linear(4,
                             6,
                             {
                                 0.10F, 0.20F, -0.05F, 0.05F,
```

```
                                    -0.15F, 0.05F, 0.25F, 0.10F,
                                    0.20F, -0.10F, 0.05F, 0.15F,
                                    0.05F, 0.10F, -0.20F, 0.25F,
                                    -0.05F, 0.15F, 0.10F, -0.10F,
                                    0.25F, -0.05F, 0.05F, 0.10F,
                                });
    layer.w_down = make_linear(6,
                               4,
                               {
                                   0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
                                   -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
                                   0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
                                   0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
                               });
    model.layers.push_back(std::move(layer));
    model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
    model.lm_head = make_linear(4,
                                5,
                                {
                                    0.20F, -0.10F, 0.05F, 0.15F,
                                    -0.05F, 0.25F, 0.10F, -0.20F,
                                    0.15F, 0.05F, -0.25F, 0.10F,
                                    -0.10F, 0.20F, 0.15F, 0.05F,
                                    0.05F, -0.15F, 0.20F, 0.25F,
                                },
                                zeros(5));
    return model;
}

}  // namespace lcqi
```

Listing 16.16 src/safetensors.cpp

```
#include <lcqi/safetensors.hpp>

#include <algorithm>
#include <cctype>
#include <cstddef>
#include <fstream>
#include <limits>
#include <stdexcept>
#include <string>
#include <string_view>

namespace lcqi {
namespace {

constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U;
constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
constexpr std::uint64_t LCQI_BYTE_BITS = 8;
```

```
constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;
constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;

class HeaderCursor {
public:
    explicit HeaderCursor(std::string_view text) : text_(text) {}

    void expect(char expected) {
        this->skip_ws();
        if (this->eof() || this->text_[this->position_] != expected) {
            throw std::runtime_error("invalid safetensors header syntax");
        }
        ++this->position_;
    }

    [[nodiscard]] bool consume(char expected) {
        this->skip_ws();
        if (!this->eof() && this->text_[this->position_] == expected) {
            ++this->position_;
            return true;
        }
        return false;
    }

    [[nodiscard]] std::string parse_string() {
        this->skip_ws();
        this->expect('"');
        std::string result;
        while (!this->eof()) {
            const char ch = this->text_[this->position_++];
            if (ch == '"') {
                return result;
            }
            if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) {
    ↪ {
                throw std::runtime_error("control character in safetensors
    ↪ string");
            }
            if (ch == '\\') {
                result.push_back(this->parse_escape());
            } else {
                result.push_back(ch);
            }
```

```
        }
        throw std::runtime_error("unterminated safetensors string");
    }

    [[nodiscard]] std::uint64_t parse_u64() {
        this->skip_ws();
        if (this->eof() ||
            !std::isdigit(static_cast<unsigned char>(this->text_[this->position_]))) {
            throw std::runtime_error("expected unsigned integer in safetensors header");
        }
        std::uint64_t value = 0;
        while (!this->eof() &&
               std::isdigit(static_cast<unsigned char>(this->text_[this->position_]))) {
            const std::uint64_t digit =
                static_cast<std::uint64_t>(this->text_[this->position_] - '0');
            if (value >
                (std::numeric_limits<std::uint64_t>::max() - digit) / LCQI_DECIMAL_BASE) {
                throw std::runtime_error("safetensors integer overflow");
            }
            value = value * LCQI_DECIMAL_BASE + digit;
            ++this->position_;
        }
        return value;
    }

    [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
        std::vector<std::int64_t> values;
        this->expect('[');
        if (this->consume(']')) {
            return values;
        }
        while (true) {
            const std::uint64_t raw = this->parse_u64();
            if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::int64_t>::max())) {
                throw std::runtime_error("safetensors shape value is too large");
            }
            values.push_back(static_cast<std::int64_t>(raw));
            if (this->consume(']')) {
                return values;
            }
            this->expect(',');
        }
    }

    [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
        std::vector<std::uint64_t> values;
```

```
        this->expect('[');
        if (this->consume(']')) {
            return values;
        }
        while (true) {
            values.push_back(this->parse_u64());
            if (this->consume(']')) {
                return values;
            }
            this->expect(',');
        }
    }

    [[nodiscard]] bool eof_after_ws() {
        this->skip_ws();
        return this->eof();
    }

private:
    std::string_view text_;
    std::size_t position_ = 0;

    [[nodiscard]] bool eof() const {
        return this->position_ >= this->text_.size();
    }

    void skip_ws() {
        while (!this->eof() &&
               std::isspace(static_cast<unsigned char>(this->text_[this->position_]))) {
            ++this->position_;
        }
    }

    [[nodiscard]] char parse_escape() {
        if (this->eof()) {
            throw std::runtime_error("unterminated safetensors escape");
        }
        const char escaped = this->text_[this->position_++];
        switch (escaped) {
            case '"':
            case '\\':
            case '/':
                return escaped;
            case 'b':
                return '\b';
            case 'f':
                return '\f';
            case 'n':
                return '\n';
            case 'r':
                return '\r';
```

```
            case 't':
                return '\t';
            default:
                throw std::runtime_error("unsupported safetensors string escape");
        }
    }
};

std::uint64_t read_le_u64(const std::string& bytes) {
    if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
        throw std::runtime_error("safetensors file is too small");
    }
    std::uint64_t value = 0;
    for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
        value |= static_cast<std::uint64_t>(
                    static_cast<unsigned char>(bytes[i]))
                 << (LCQI_BYTE_BITS * i);
    }
    return value;
}

std::string read_file_bytes(const std::filesystem::path& path) {
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open safetensors file: " + path.string());
    }
    input.seekg(0, std::ios::end);
    const std::streamoff size = input.tellg();
    if (size < 0) {
        throw std::runtime_error("cannot determine safetensors file size");
    }
    input.seekg(0, std::ios::beg);
    std::string bytes(static_cast<std::size_t>(size), '\0');
    if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize>(bytes.size()))) {
        throw std::runtime_error("cannot read safetensors file");
    }
    return bytes;
}

void parse_metadata_value(HeaderCursor& cursor,
                          SafeTensorManifest& manifest,
                          const std::string& key) {
    if (key == "__metadata__") {
        cursor.expect('{');
        if (cursor.consume('}')) {
            return;
        }
        while (true) {
            const std::string metadata_key = cursor.parse_string();
```

```
            cursor.expect(':');
            const std::string metadata_value = cursor.parse_string();
            manifest.metadata.emplace_back(metadata_key, metadata_value);
            if (cursor.consume('}')) {
                return;
            }
            cursor.expect(',');
        }
    }
    throw std::runtime_error("unexpected safetensors metadata value");
}

SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& name) {
    SafeTensorEntry entry;
    entry.name = name;
    bool saw_dtype = false;
    bool saw_shape = false;
    bool saw_offsets = false;

    cursor.expect('{');
    if (cursor.consume('}')) {
        throw std::runtime_error("empty safetensors tensor entry");
    }
    while (true) {
        const std::string field = cursor.parse_string();
        cursor.expect(':');
        if (field == "dtype") {
            entry.dtype = cursor.parse_string();
            saw_dtype = true;
        } else if (field == "shape") {
            entry.shape = cursor.parse_i64_array();
            saw_shape = true;
        } else if (field == "data_offsets") {
            const std::vector<std::uint64_t> offsets = cursor.parse_u64_array();
            if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
                throw std::runtime_error("safetensors data_offsets must contain two values");
            }
            entry.data_begin = offsets[0];
            entry.data_end = offsets[1];
            saw_offsets = true;
        } else {
            throw std::runtime_error("unsupported safetensors tensor field: " + field);
        }
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }
```

```
263
264     if (!saw_dtype || !saw_shape || !saw_offsets) {
265         throw std::runtime_error("safetensors tensor entry is missing required ↩
    ↪ fields");
266     }
267     if (entry.data_end < entry.data_begin) {
268         throw std::runtime_error("safetensors tensor offset range is invalid");
269     }
270     return entry;
271 }
272
273 void parse_header(std::string_view header, SafeTensorManifest& manifest) {
274     HeaderCursor cursor(header);
275     cursor.expect('{');
276     if (cursor.consume('}')) {
277         throw std::runtime_error("safetensors header is empty");
278     }
279
280     while (true) {
281         const std::string key = cursor.parse_string();
282         cursor.expect(':');
283         if (key == "__metadata__") {
284             parse_metadata_value(cursor, manifest, key);
285         } else {
286             manifest.tensors.push_back(parse_tensor_entry(cursor, key));
287         }
288         if (cursor.consume('}')) {
289             break;
290         }
291         cursor.expect(',');
292     }
293
294     if (!cursor.eof_after_ws()) {
295         throw std::runtime_error("trailing data after safetensors header");
296     }
297 }
298
299 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);
300
301 void validate_manifest(const SafeTensorManifest& manifest,
302                        std::uint64_t file_size) {
303     const std::uint64_t payload_size = file_size - manifest.data_start_offset;
304     for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
305         const SafeTensorEntry& left = manifest.tensors[i];
306         if (left.name.empty() || left.dtype.empty()) {
307             throw std::runtime_error("safetensors tensor has empty name or ↩
    ↪ dtype");
308         }
309         if (left.data_end > payload_size) {
310             throw std::runtime_error("safetensors tensor data_offsets exceed ↩
    ↪ payload size");
311         }
```

```
312         const std::uint64_t expected_bytes = expected_tensor_bytes(left);
313         if (left.byte_size() != expected_bytes) {
314             throw std::runtime_error("safetensors tensor byte size does not match dtype and shape");
315         }
316         for (const std::int64_t dim : left.shape) {
317             if (dim < 0) {
318                 throw std::runtime_error("safetensors shape dimension is negative");
319             }
320         }
321         for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {
322             const SafeTensorEntry& right = manifest.tensors[j];
323             if (left.name == right.name) {
324                 throw std::runtime_error("duplicate safetensors tensor name");
325             }
326             const bool overlaps =
327                 left.data_begin < right.data_end && right.data_begin < left.data_end;
328             if (overlaps) {
329                 throw std::runtime_error("overlapping safetensors tensor data range");
330             }
331         }
332     }
333 }
334
335 std::uint64_t dtype_byte_size(std::string_view dtype) {
336     if (dtype == "F64") {
337         return LCQI_DTYPE_BYTES_F64;
338     }
339     if (dtype == "F32") {
340         return LCQI_DTYPE_BYTES_F32;
341     }
342     if (dtype == "F16" || dtype == "BF16") {
343         return LCQI_DTYPE_BYTES_F16;
344     }
345     if (dtype == "I64" || dtype == "U64") {
346         return LCQI_DTYPE_BYTES_I64;
347     }
348     if (dtype == "I32" || dtype == "U32") {
349         return LCQI_DTYPE_BYTES_I32;
350     }
351     if (dtype == "I16" || dtype == "U16") {
352         return LCQI_DTYPE_BYTES_I16;
353     }
354     if (dtype == "I8") {
355         return LCQI_DTYPE_BYTES_I8;
356     }
357     if (dtype == "U8" || dtype == "BOOL") {
358         return LCQI_DTYPE_BYTES_U8;
359     }
```

```
360     throw std::runtime_error("unsupported safetensors dtype: " + std::string(↩
    ↪ dtype));
361 }
362
363 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
364     std::uint64_t element_count = 1;
365     for (const std::int64_t dim : entry.shape) {
366         if (dim < 0) {
367             throw std::runtime_error("safetensors shape dimension is negative")↩
    ↪ ;
368         }
369         const std::uint64_t extent = static_cast<std::uint64_t>(dim);
370         if (extent != 0 &&
371             element_count > std::numeric_limits<std::uint64_t>::max() / extent)↩
    ↪  {
372             throw std::runtime_error("safetensors shape element count overflow"↩
    ↪ );
373         }
374         element_count *= extent;
375     }
376     const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
377     if (dtype_bytes != 0 &&
378         element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes↩
    ↪ ) {
379         throw std::runtime_error("safetensors tensor byte size overflow");
380     }
381     return element_count * dtype_bytes;
382 }
383
384 }  // namespace
385
386 std::uint64_t SafeTensorEntry::byte_size() const {
387     return this->data_end - this->data_begin;
388 }
389
390 const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) ↩
    ↪ const {
391     const auto found = std::find_if(
392         this->tensors.begin(),
393         this->tensors.end(),
394         [name](const SafeTensorEntry& entry) {
395             return entry.name == name;
396         });
397     if (found == this->tensors.end()) {
398         return nullptr;
399     }
400     return &(*found);
401 }
402
403 SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path)↩
    ↪  {
404     const std::string bytes = read_file_bytes(path);
```

```
    const std::uint64_t header_size = read_le_u64(bytes);
    if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
        throw std::runtime_error("safetensors header size is invalid");
    }
    const std::uint64_t data_start_offset =
        static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size;
    if (data_start_offset > bytes.size()) {
        throw std::runtime_error("safetensors header extends beyond file size");
    }

    SafeTensorManifest manifest;
    manifest.header_size = header_size;
    manifest.data_start_offset = data_start_offset;
    const std::string_view header(
        bytes.data() + LCQI_SAFETENSORS_PREFIX_BYTES,
        static_cast<std::size_t>(header_size));
    parse_header(header, manifest);
    validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
    return manifest;
}

}  // namespace lcqi
```

Listing 16.17 src/tokenizer.cpp

```
#include <lcqi/tokenizer.hpp>

#include <cstddef>
#include <fstream>
#include <stdexcept>
#include <string>
#include <string_view>

namespace lcqi {
namespace {

constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 14695981039346656037ULL;
constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;

std::string read_key(std::istream& input) {
    std::string key;
    if (!(input >> key)) {
        throw std::runtime_error("unexpected end of tokenizer file");
    }
    return key;
}

void expect_key(std::istream& input, const std::string& expected) {
    const std::string key = read_key(input);
```

```
    if (key != expected) {
        throw std::runtime_error("expected tokenizer key '" + expected + "', got '" + key + "'");
    }
}

std::int32_t read_i32(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::int32_t value = 0;
    if (!(input >> value)) {
        throw std::runtime_error("invalid tokenizer int32 value for key '" + key + "'");
    }
    return value;
}

std::string read_string(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::string value;
    if (!(input >> value)) {
        throw std::runtime_error("invalid tokenizer string value for key '" + key + "'");
    }
    return value;
}

std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view text) {
    for (const TokenEntry& entry : tokenizer.vocab) {
        if (entry.text == text) {
            return entry.id;
        }
    }
    return LCQI_INVALID_TOKEN_ID;
}

void validate_tokenizer(const TokenizerModel& tokenizer) {
    if (tokenizer.vocab.empty()) {
        throw std::runtime_error("tokenizer vocab is empty");
    }
    for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
        const TokenEntry& left = tokenizer.vocab[i];
        if (left.id < 0) {
            throw std::runtime_error("tokenizer token id must be non-negative");
        }
        for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {
            const TokenEntry& right = tokenizer.vocab[j];
            if (left.id == right.id) {
                throw std::runtime_error("duplicate tokenizer token id");
            }
            if (left.text == right.text) {
```

```
                throw std::runtime_error("duplicate tokenizer token text");
            }
        }
    }
    if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
        lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
        throw std::runtime_error("tokenizer BOS id does not match vocab");
    }
    if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
        lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
        throw std::runtime_error("tokenizer EOS id does not match vocab");
    }
}

std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
    hash ^= static_cast<std::uint64_t>(value);
    hash *= LCQI_FNV_PRIME;
    return hash;
}

std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
    for (const unsigned char value : text) {
        hash = hash_byte(hash, value);
    }
    return hash;
}

std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
    const std::uint32_t bits = static_cast<std::uint32_t>(value);
    for (std::int32_t shift = 0; shift < 32; shift += 8) {
        hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0↩
    ↪ xFFU));
    }
    return hash;
}

}  // namespace

TokenizerModel load_tokenizer(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open tokenizer file: " + path.string()↩
    ↪ );
    }

    expect_key(input, "LCQI_TOKENIZER_V1");

    TokenizerModel tokenizer;
    tokenizer.bos_id = read_i32(input, "bos_id");
    tokenizer.eos_id = read_i32(input, "eos_id");
    tokenizer.unk_id = read_i32(input, "unk_id");
    tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
```

```cpp
    const std::int32_t vocab_size = read_i32(input, "vocab_size");
    if (vocab_size <= 0) {
        throw std::runtime_error("tokenizer vocab_size must be positive");
    }

    tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
    expect_key(input, "vocab");
    for (std::int32_t i = 0; i < vocab_size; ++i) {
        TokenEntry entry;
        if (!(input >> entry.id >> entry.text)) {
            throw std::runtime_error("invalid tokenizer vocab entry");
        }
        tokenizer.vocab.push_back(entry);
    }

    validate_tokenizer(tokenizer);
    return tokenizer;
}

std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
                                        std::string_view prompt) {
    std::vector<std::int32_t> ids;
    if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
        ids.push_back(tokenizer.bos_id);
    }

    std::size_t begin = 0;
    while (begin < prompt.size()) {
        while (begin < prompt.size() && prompt[begin] == ' ') {
            ++begin;
        }
        if (begin >= prompt.size()) {
            break;
        }
        std::size_t end = begin;
        while (end < prompt.size() && prompt[end] != ' ') {
            ++end;
        }
        const std::string_view token = prompt.substr(begin, end - begin);
        const std::int32_t token_id = lookup_token_id(tokenizer, token);
        if (token_id == LCQI_INVALID_TOKEN_ID) {
            if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
                throw std::runtime_error("tokenizer encountered unknown token")↩
    ↪ ;
            }
            ids.push_back(tokenizer.unk_id);
        } else {
            ids.push_back(token_id);
        }
        begin = end;
    }

```

```
    if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
        ids.push_back(tokenizer.eos_id);
    }
    return ids;
}

std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {
    std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
    hash = hash_string(hash, tokenizer.chat_template_hash);
    hash = hash_i32(hash, tokenizer.bos_id);
    hash = hash_i32(hash, tokenizer.eos_id);
    hash = hash_i32(hash, tokenizer.unk_id);
    for (const TokenEntry& entry : tokenizer.vocab) {
        hash = hash_string(hash, entry.text);
        hash = hash_i32(hash, entry.id);
    }
    return hash;
}

}  // namespace lcqi
```

四、命令行、账本、Trace 和 Benchmark

Listing 16.18 src/main.cpp

```
#include <lcqi/inference.hpp>
#include <lcqi/model.hpp>

#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <filesystem>
#include <iostream>
#include <span>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;

std::vector<float> parse_input(int argc, char** argv, std::int32_t expected_size) {
    if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
        throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...> [repeat]");
    }
    std::vector<float> input(static_cast<std::size_t>(expected_size));
```

```
    for (std::int32_t i = 0; i < expected_size; ++i) {
        input[static_cast<std::size_t>(i)] =
            std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
    }
    return input;
}

std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
    const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;
    if (argc <= repeat_index) {
        return LCQI_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[repeat_index]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

}  // namespace

int main(int argc, char** argv) {
    try {
        if (argc < LCQI_EXPECTED_ARGC_MIN) {
            throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats> [repeat]");
        }

        const std::filesystem::path model_path = argv[1];
        const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
        const std::vector<float> input = parse_input(argc, argv, model.input_size);
        const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);

        const lcqi::InferenceResult result = lcqi::run_inference(model, input);
        std::cout << "predicted_class " << result.predicted_class << "\n";
        std::cout << "logits";
        for (const float value : result.logits) {
            std::cout << ' ' << value;
        }
        std::cout << "\n";

        const auto begin = std::chrono::steady_clock::now();
        std::int32_t checksum = 0;
        for (std::int32_t i = 0; i < repeat; ++i) {
            checksum += lcqi::run_inference(model, input).predicted_class;
        }
        const auto end = std::chrono::steady_clock::now();
        const std::chrono::duration<double> elapsed = end - begin;
        const double average_us =
```

```
            elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
        std::cout << "repeat " << repeat << "\n";
        std::cout << "average_us " << average_us << "\n";
        std::cout << "checksum " << checksum << "\n";
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

Listing 16.19 src/bench.cpp

```
#include <lcqi/int8_kernels.hpp>

#include <algorithm>
#include <cstdlib>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
constexpr int LCQI_REPEAT_ARG_INDEX = 1;
constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;

const char* target_arch() noexcept {
#if defined(__x86_64__) || defined(_M_X64)
    return LCQI_TARGET_ARCH_X86_64;
#elif defined(__i386__) || defined(_M_IX86)
    return LCQI_TARGET_ARCH_I386;
#else
    return LCQI_TARGET_ARCH_UNKNOWN;
#endif
}

std::int32_t parse_repeat(int argc, char** argv) {
    if (argc <= LCQI_REPEAT_ARG_INDEX) {
        return LCQI_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[LCQI_REPEAT_ARG_INDEX]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
```

```
    }
    return repeat;
}

std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
    return {
        {128, 128, 16, repeat},
        {256, 256, 16, repeat},
        {512, 512, 16, repeat},
        {513, 257, 16, repeat},
        {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
                                  repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
        {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
         std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
    };
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const std::int32_t repeat = parse_repeat(argc, argv);
        std::cout
            << "target_arch,input_size,output_size,output_block,repeat,backend,available,"
               "average_us,max_abs_diff,checksum\n";
        for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(repeat)) {
            const lcqi::KernelBenchmarkResult result =
                lcqi::benchmark_linear_i8_case(benchmark_case);
            for (const lcqi::KernelBackendBenchmark& backend : result.backends) {
                std::cout << target_arch() << ','
                          << result.benchmark_case.input_size << ','
                          << result.benchmark_case.output_size << ','
                          << result.benchmark_case.output_block_size << ','
                          << result.benchmark_case.repeat << ','
                          << backend.name << ','
                          << (backend.available ? 1 : 0) << ','
                          << backend.average_us << ','
                          << backend.max_abs_diff << ','
                          << backend.checksum << '\n';
            }
        }
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

Listing 16.20 src/trace.cpp

```cpp
#include <lcqi/reference_decoder.hpp>
#include <lcqi/tokenizer.hpp>

#include <algorithm>
#include <array>
#include <cstdlib>
#include <exception>
#include <filesystem>
#include <iostream>
#include <span>
#include <sstream>
#include <string>
#include <string_view>

namespace {

constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;

std::filesystem::path tokenizer_fixture_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_tokenizer.txt";
}

std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel
    & tokenizer,
                                                     std::string_view prompt) {
    std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer,
    prompt);
    if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
        full_tokens.front() != LCQI_TRACE_BOS_ID ||
        full_tokens.back() != LCQI_TRACE_EOS_ID) {
        throw std::runtime_error("unexpected tokenizer fixture ids");
    }

    std::vector<std::int32_t> decoder_tokens(
        full_tokens.begin() + 1,
        full_tokens.end() - 1);
    if (!std::equal(decoder_tokens.begin(),
                    decoder_tokens.end(),
                    LCQI_TRACE_TOKEN_IDS.begin(),
                    LCQI_TRACE_TOKEN_IDS.end())) {
        throw std::runtime_error("unexpected decoder token ids");
    }
    return decoder_tokens;
}

```

```cpp
std::string join_ints(std::span<const std::int32_t> values) {
    std::ostringstream stream;
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (index != 0) {
            stream << 'x';
        }
        stream << values[index];
    }
    return stream.str();
}

std::string join_int_values(std::span<const std::int32_t> values) {
    std::ostringstream stream;
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (index != 0) {
            stream << ';';
        }
        stream << values[index];
    }
    return stream.str();
}

std::int32_t checksum_ints(std::span<const std::int32_t> values) {
    std::int32_t result = 0;
    for (const std::int32_t value : values) {
        result += value;
    }
    return result;
}

std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
    std::int32_t result = 0;
    for (const std::int32_t value : values) {
        result = std::max(result, std::abs(value));
    }
    return result;
}

std::string join_floats(std::span<const float> values) {
    std::ostringstream stream;
    const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);
    for (std::size_t index = 0; index < count; ++index) {
        if (index != 0) {
            stream << ';';
        }
        stream << values[index];
    }
    if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
        stream << ";...";
    }
    return stream.str();
}
```

```
102
103 void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
104                       const lcqi::TokenizerModel& tokenizer,
105                       std::span<const std::int32_t> full_tokens) {
106     std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,↩
    ↪ checksum,max_abs,values\n";
107     std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
108               << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
109     std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
110               << ',' << full_tokens.size() << ",1,i32,contiguous,"
111               << checksum_ints(full_tokens) << ','
112               << max_abs_ints(full_tokens) << ','
113               << join_int_values(full_tokens) << '\n';
114     std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
115               << ",scalar,scalar,u64,fnv1a,"
116               << tokenizer_contract_hash(tokenizer) << ",0,"
117               << tokenizer.chat_template_hash << '\n';
118     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↩
    ↪  {
119         std::cout << checkpoint.name << ','
120                   << checkpoint.token_position << ','
121                   << checkpoint.layer_id << ','
122                   << join_ints(checkpoint.shape) << ','
123                   << join_ints(checkpoint.stride) << ','
124                   << checkpoint.dtype << ','
125                   << checkpoint.layout << ','
126                   << checkpoint.checksum << ','
127                   << checkpoint.max_abs << ','
128                   << join_floats(checkpoint.values) << '\n';
129     }
130     std::cout << "sampler_argmax,"
131               << LCQI_TRACE_TOKEN_IDS.size() - 1 << ','
132               << LCQI_TRACE_METADATA_LAYER
133               << ',' << trace.result.logits.size() << ",1,f32,argmax,"
134               << "0,0,token=" << trace.result.predicted_token << '\n';
135     std::cout << "generated_token,"
136               << LCQI_TRACE_TOKEN_IDS.size() << ','
137               << LCQI_TRACE_METADATA_LAYER
138               << ",scalar,scalar,i32,generated,"
139               << trace.result.predicted_token << ','
140               << trace.result.predicted_token << ','
141               << trace.result.predicted_token << '\n';
142 }
143
144 }  // namespace
145
146 int main() {
147     try {
148         const lcqi::TokenizerModel tokenizer =
149             lcqi::load_tokenizer(tokenizer_fixture_path());
150         const std::vector<std::int32_t> full_tokens =
151             lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
```

```
        const std::vector<std::int32_t> decoder_tokens =
            decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
        const lcqi::ReferenceDecoderModel model = lcqi::↩
    ↪ make_tiny_reference_decoder_model();
        const lcqi::ReferenceDecodeTraceResult trace =
            lcqi::run_reference_decode_with_trace(model, decoder_tokens);
        print_trace_csv(trace, tokenizer, full_tokens);
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

Listing 16.21 src/ledger.cpp

```
#include <cstdlib>
#include <cstdint>
#include <exception>
#include <iomanip>
#include <iostream>
#include <string_view>

namespace {

constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
constexpr double LCQI_FLOPS_PER_MAC = 2.0;
constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;

struct DTypeLedger {
    std::string_view name;
    double weight_gb_per_token = 0.0;
    double kv_element_bytes = 0.0;
};

constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
    {"fp16", 14.0, 2.0},
    {"int8", 7.0, 2.0},
    {"int4", 3.7, 2.0},
};

constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
    50.0,
    100.0,
    200.0,
```

```
    600.0,
    1000.0,
    1600.0,
};

double linear_macs_per_layer() noexcept {
    const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
    const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
    const double qkv = hidden * (hidden + 2.0 * kv_width);
    const double output_projection = hidden * hidden;
    const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
    return qkv + output_projection + mlp;
}

double attention_macs_per_layer() noexcept {
    return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
           LCQI_SEVENB_HEAD_DIM;
}

double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
    return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
           LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
}

double kv_read_gb_per_decode_token(double element_bytes) noexcept {
    const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
                         kv_bytes_per_token_all_layers(element_bytes);
    return bytes / LCQI_BYTES_PER_GB;
}

double kv_capacity_mib(double element_bytes, std::int32_t context_length) noexcept {
    const double bytes = static_cast<double>(context_length) *
                         kv_bytes_per_token_all_layers(element_bytes);
    return bytes / LCQI_BYTES_PER_MIB;
}

void print_model_rows() {
    const double linear_macs = linear_macs_per_layer();
    const double attention_macs = attention_macs_per_layer();
    const double total_flops =
        (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT * LCQI_FLOPS_PER_MAC;
    std::cout << "section,item,value,unit,notes\n";
    std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B fixture\n";
    std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";
    std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,GQA query heads\n";
    std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,GQA KV heads\n";
    std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per
```

```
    ↪ head\n";
    std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs,↩
    ↪ decode token\n";
    std::cout << "compute,attention_macs_per_layer," << attention_macs
              << ",macs,context=4096\n";
    std::cout << "compute,total_decode_flops_per_token," << total_flops
              << ",flops,linear plus attention\n";
}

void print_kv_rows() {
    for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
        std::cout << "kv,fp16_capacity_context_" << context << ','
                  << kv_capacity_mib(2.0, context)
                  << ",MiB,single session\n";
    }
}

void print_dtype_rows() {
    for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
        const double kv_read_gb = kv_read_gb_per_decode_token(dtype.↩
    ↪ kv_element_bytes);
        const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
        std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
                  << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
        std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
                  << kv_read_gb << ",GB/token,context=4096\n";
        std::cout << "decode_bytes," << dtype.name << "_total_gb,"
                  << total_gb << ",GB/token,weight plus KV\n";
        for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
            std::cout << "bandwidth_limit," << dtype.name << "_at_"
                      << static_cast<std::int32_t>(bandwidth) << "_gbps,"
                      << bandwidth / total_gb
                      << ",tokens/s,bandwidth-only upper bound\n";
        }
    }
}

}  // namespace

int main() {
    try {
        std::cout << std::fixed << std::setprecision(3);
        print_model_rows();
        print_kv_rows();
        print_dtype_rows();
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

Listing 16.22 src/serving_probe.cpp

```cpp
#include <algorithm>
#include <cstdlib>
#include <cstdint>
#include <exception>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <string_view>
#include <vector>

namespace {

constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;

struct ProbeRequest {
    std::string_view request_id;
    std::int32_t prompt_tokens = 0;
    std::int32_t max_new_tokens = 0;
    bool cancel_after_prefill = false;
};

struct ProbeResult {
    std::string_view request_id;
    std::int32_t accepted = 0;
    std::string_view rejection_reason;
    std::int32_t prompt_tokens = 0;
    std::int32_t reserved_tokens = 0;
    double kv_reserved_mib = 0.0;
    double queue_wait_ms = 0.0;
    double prefill_ms = 0.0;
    double ttft_ms = 0.0;
    double decode_step_p95_ms = 0.0;
    double tpot_ms = 0.0;
    std::int32_t completion_tokens = 0;
    std::string_view finish_reason;
    double cancel_reclaim_ms = 0.0;
};

```

```cpp
const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
    {"req_short", 32, 4, false},
    {"req_rag", 512, 8, false},
    {"req_cancel", 128, 16, true},
    {"req_too_long", 4096, 512, false},
};

double kv_mib_for_tokens(std::int32_t tokens) noexcept {
    const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT * 2.0 *
                         LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
                         LCQI_PROBE_KV_ELEMENT_BYTES;
    return bytes / LCQI_BYTES_PER_MIB;
}

double percentile(std::vector<double> values, double fraction) {
    if (values.empty()) {
        return 0.0;
    }
    std::sort(values.begin(), values.end());
    const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
    const std::size_t index =
        std::min(max_index,
                 static_cast<std::size_t>(
                     std::ceil(fraction * static_cast<double>(max_index))));
    return values[index];
}

std::vector<ProbeResult> run_probe() {
    std::vector<ProbeResult> results;
    double reserved_kv_mib = 0.0;
    std::int32_t accepted_before = 0;
    for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
        ProbeResult result;
        result.request_id = request.request_id;
        result.prompt_tokens = request.prompt_tokens;
        result.reserved_tokens = request.prompt_tokens + request.max_new_tokens;
        result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);
        const double static_session_mib =
            result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +
            LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
        if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB) {
            result.accepted = 0;
            result.rejection_reason = "memory_budget_exceeded";
            result.finish_reason = "rejected";
            results.push_back(result);
            continue;
        }

        reserved_kv_mib += result.kv_reserved_mib;
```

```
        result.accepted = 1;
        result.rejection_reason = "none";
        result.queue_wait_ms = static_cast<double>(accepted_before) *
                               LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
        result.prefill_ms = static_cast<double>(request.prompt_tokens) *
                            LCQI_PROBE_PREFILL_MS_PER_TOKEN;
        result.ttft_ms = result.queue_wait_ms + result.prefill_ms + LCQI_PROBE_DECODE_STEP_MS;
        result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
                                    static_session_mib / 512.0;
        result.tpot_ms = result.decode_step_p95_ms;
        if (request.cancel_after_prefill) {
            result.completion_tokens = 0;
            result.finish_reason = "cancelled";
            result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
            reserved_kv_mib -= result.kv_reserved_mib;
        } else {
            result.completion_tokens = request.max_new_tokens;
            result.finish_reason = "max_tokens";
        }
        ++accepted_before;
        results.push_back(result);
    }
    return results;
}

void print_request_rows(const std::vector<ProbeResult>& results) {
    std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,"
                 "kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,"
                 "tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,"
                 "metric_value\n";
    for (const ProbeResult& result : results) {
        std::cout << "request," << result.request_id << ','
                  << result.accepted << ','
                  << result.rejection_reason << ','
                  << result.prompt_tokens << ','
                  << result.reserved_tokens << ','
                  << result.kv_reserved_mib << ','
                  << result.queue_wait_ms << ','
                  << result.prefill_ms << ','
                  << result.ttft_ms << ','
                  << result.decode_step_p95_ms << ','
                  << result.tpot_ms << ','
                  << result.completion_tokens << ','
                  << result.finish_reason << ','
                  << result.cancel_reclaim_ms << ",,\n";
    }
}

```

```cpp
void print_summary_row(const std::vector<ProbeResult>& results) {
    std::vector<double> ttft_values;
    std::vector<double> queue_values;
    double accepted = 0.0;
    double rejected = 0.0;
    double cancelled = 0.0;
    double reserved_peak_mib = 0.0;
    double running_reserved_mib = 0.0;
    for (const ProbeResult& result : results) {
        if (result.accepted == 0) {
            rejected += 1.0;
            continue;
        }
        accepted += 1.0;
        ttft_values.push_back(result.ttft_ms);
        queue_values.push_back(result.queue_wait_ms);
        running_reserved_mib += result.kv_reserved_mib;
        reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
        if (result.finish_reason == "cancelled") {
            cancelled += 1.0;
            running_reserved_mib -= result.kv_reserved_mib;
        }
    }
    const double total = accepted + rejected;
    const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "accepted_count," << accepted << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,"
              << LCQI_PROBE_CANCEL_RECLAIM_MS << ",cancel_count," << cancelled
    << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "rejected_count," << rejected << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "rejection_rate," << rejection_rate << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n'
    ;
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';
}

}  // namespace

int main() {
    try {
        std::cout << std::fixed << std::setprecision(3);
        const std::vector<ProbeResult> results = run_probe();
        print_request_rows(results);
        print_summary_row(results);
        return EXIT_SUCCESS;
```

```
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

Listing 16.23 src/onednn_bench.cpp

```
#include <dnnl.hpp>

#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <iostream>
#include <numeric>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <vector>

namespace {

constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
constexpr std::int64_t LCQI_ONEDNN_K = 4096;
constexpr std::int64_t LCQI_ONEDNN_O = 4096;
constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
constexpr int LCQI_REPEAT_ARG_INDEX = 1;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
constexpr float LCQI_INPUT_SCALE = 0.125F;
constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
constexpr std::int32_t LCQI_INPUT_CENTER = 8;
constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
constexpr float LCQI_WEIGHT_SCALE = 0.03125F;

std::int32_t parse_repeat(int argc, char** argv) {
    if (argc <= LCQI_REPEAT_ARG_INDEX) {
        return LCQI_ONEDNN_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↩
    ↪ LCQI_REPEAT_ARG_INDEX]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

std::vector<float> make_input() {
    std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
    for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
        input[static_cast<std::size_t>(index)] =
```

```
            static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER) *
            LCQI_INPUT_SCALE;
    }
    return input;
}

std::vector<float> make_weights_kn() {
    std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K * LCQI_ONEDNN_O));
    for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
        for (std::int64_t out = 0; out < LCQI_ONEDNN_O; ++out) {
            const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
            const float value =
                static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
                                   LCQI_WEIGHT_CENTER) *
                LCQI_WEIGHT_SCALE;
            weights[static_cast<std::size_t>(k * LCQI_ONEDNN_O + out)] = value;
        }
    }
    return weights;
}

float checksum(const std::vector<float>& values) {
    return std::accumulate(values.begin(), values.end(), 0.0F);
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const std::int32_t repeat = parse_repeat(argc, argv);
        std::vector<float> input = make_input();
        std::vector<float> weights = make_weights_kn();
        std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH * LCQI_ONEDNN_O),
                                  0.0F);

        dnnl::engine engine(dnnl::engine::kind::cpu, 0);
        dnnl::stream stream(engine);

        const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH, LCQI_ONEDNN_K};
        const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_O};
        const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, LCQI_ONEDNN_O};

        const dnnl::memory::desc source_desc(
            source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag::ab);
        const dnnl::memory::desc weight_desc(
            weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag
```

```
    ↪ ::ab);
90          const dnnl::memory::desc output_desc(
91              output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↩
    ↪ ::ab);
92
93          dnnl::memory source_memory(source_desc, engine, input.data());
94          dnnl::memory weight_memory(weight_desc, engine, weights.data());
95          dnnl::memory output_memory(output_desc, engine, output.data());
96
97          dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, ↩
    ↪ weight_desc, output_desc);
98          dnnl::matmul matmul(matmul_desc);
99          const std::unordered_map<int, dnnl::memory> arguments = {
100             {DNNL_ARG_SRC, source_memory},
101             {DNNL_ARG_WEIGHTS, weight_memory},
102             {DNNL_ARG_DST, output_memory},
103         };
104
105         for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
106             matmul.execute(stream, arguments);
107             stream.wait();
108         }
109
110         const auto begin = std::chrono::steady_clock::now();
111         for (std::int32_t index = 0; index < repeat; ++index) {
112             matmul.execute(stream, arguments);
113             stream.wait();
114         }
115         const auto end = std::chrono::steady_clock::now();
116         const std::chrono::duration<double> elapsed = end - begin;
117         const double average_us =
118             elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double↩
    ↪ >(repeat);
119
120         std::cout << "library,input_size,output_size,dtype,repeat,average_us,↩
    ↪ checksum\n";
121         std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_O
122                   << ",f32," << repeat << ',' << average_us << ','
123                   << checksum(output) << '\n';
124         return EXIT_SUCCESS;
125     } catch (const std::exception& error) {
126         std::cerr << "error: " << error.what() << '\n';
127         return EXIT_FAILURE;
128     }
129 }
```

五、测试和模型 Fixture

Listing 16.24 tests/lcqi_tests.cpp

```
1 #include <lcqi/inference.hpp>
```

```cpp
#include <lcqi/int8_kernels.hpp>
#include <lcqi/model.hpp>
#include <lcqi/reference_decoder.hpp>
#include <lcqi/safetensors.hpp>
#include <lcqi/tokenizer.hpp>

#include <array>
#include <cmath>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <span>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;
constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;
constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> ↩
    ↪ LCQI_REFERENCE_DECODER_LOGITS{
    0.352516979F,
    -0.126884326F,
    0.0797837451F,
    -0.29797405F,
    0.127030417F,
};
constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 1345290284538833373 4ULL;
constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;
constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
```

```cpp
constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;

struct KernelShapeCase {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
};

constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
    {17, 19, 8},
    {33, 7, 8},
    {64, 32, 16},
    {65, 31, 16},
}};

void require(bool condition, const char* message) {
    if (!condition) {
        throw std::runtime_error(message);
    }
}

void require_close(float actual, float expected, const char* message) {
    if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
        throw std::runtime_error(message);
    }
}

std::filesystem::path test_model_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_mlp_i8.txt";
}

std::filesystem::path test_tokenizer_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_tokenizer.txt";
}

std::filesystem::path temp_file_path(const std::string& name) {
    return std::filesystem::temp_directory_path() / name;
}

void write_le_u64(std::ofstream& output, std::uint64_t value) {
    for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
        const char byte = static_cast<char>(
            (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
        output.write(&byte, 1);
    }
}
```

```cpp

void write_safetensors_fixture(const std::filesystem::path& path,
                               const std::string& header,
                               std::uint64_t payload_bytes) {
    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create safetensors test fixture");
    }
    write_le_u64(output, static_cast<std::uint64_t>(header.size()));
    output.write(header.data(), static_cast<std::streamsize>(header.size()));
    for (std::uint64_t i = 0; i < payload_bytes; ++i) {
        const char byte = static_cast<char>(i & 0xFFU);
        output.write(&byte, 1);
    }
}

void test_tiny_mlp() {
    const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
    require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
    require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch");
    require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch");

    const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
    const lcqi::InferenceResult result = lcqi::run_inference(model, input);

    require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected predicted class");
    require(result.logits.size() == static_cast<std::size_t>(LCQI_TEST_OUTPUT_SIZE),
            "unexpected logits size");
    require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");
    require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");

    std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), 0.0F);
    lcqi::linear_i8(model.hidden, input, hidden);
    lcqi::relu_inplace(hidden);
    require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
    require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");
    require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
}

void test_tokenizer_contract() {
    const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(test_tokenizer_path());
    require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
    require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
    require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
    require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
            "tokenizer template hash mismatch");
```

```
151
152     const std::vector<std::int32_t> token_ids =
153         lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
154     const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
155     require(token_ids == expected, "tokenizer prompt ids mismatch");
156     require(lcqi::tokenizer_contract_hash(tokenizer) == ↩
    ↪ LCQI_TOKENIZER_HASH_EXPECTED,
157             "tokenizer contract hash mismatch");
158 }
159
160 void test_tokenizer_rejects_unknown_without_unk() {
161     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↩
    ↪ test_tokenizer_path());
162     bool threw = false;
163     try {
164         static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"))↩
    ↪ ;
165     } catch (const std::runtime_error&) {
166         threw = true;
167     }
168     require(threw, "tokenizer should reject unknown token when unk id is absent↩
    ↪ ");
169 }
170
171 void test_safetensors_manifest_contract() {
172     const std::filesystem::path path =
173         temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
174     const std::string header =
175         R"({"__metadata__":{"format":"lcqi-fixture"},"model.embed_tokens.weight↩
    ↪ ":{"dtype":"F32","shape":[2,3],"data_offsets":[0,24]},"model.layers.0.↩
    ↪ attn.q_proj.weight":{"dtype":"F16","shape":[4,3],"data_offsets"↩
    ↪ :[24,48]}})";
176     write_safetensors_fixture(path, header, ↩
    ↪ LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);
177
178     const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(↩
    ↪ path);
179     std::filesystem::remove(path);
180
181     require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
182             "safetensors header size mismatch");
183     require(manifest.data_start_offset ==
184                 static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
185                     static_cast<std::uint64_t>(header.size()),
186             "safetensors data start mismatch");
187     require(manifest.metadata.size() == 1, "safetensors metadata count mismatch↩
    ↪ ");
188     require(manifest.metadata[0].first == "format" &&
189                 manifest.metadata[0].second == "lcqi-fixture",
190             "safetensors metadata mismatch");
191     require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");
192
```

```
    const lcqi::SafeTensorEntry* embedding =
        manifest.find_tensor("model.embed_tokens.weight");
    require(embedding != nullptr, "safetensors embedding tensor missing");
    require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
    require(embedding->shape == std::vector<std::int64_t>({2, 3}),
            "safetensors embedding shape mismatch");
    require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
                embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
            "safetensors embedding offset mismatch");
    require(embedding->byte_size() ==
                LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
            "safetensors embedding byte size mismatch");

    const lcqi::SafeTensorEntry* q_proj =
        manifest.find_tensor("model.layers.0.attn.q_proj.weight");
    require(q_proj != nullptr, "safetensors q projection tensor missing");
    require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
    require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
                q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
            "safetensors q projection offset mismatch");
}

void test_safetensors_rejects_bad_offsets() {
    const std::filesystem::path path =
        temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
    const std::string header =
        R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,32]}})";
    write_safetensors_fixture(path, header, 4);

    bool threw = false;
    try {
        static_cast<void>(lcqi::load_safetensors_manifest(path));
    } catch (const std::runtime_error&) {
        threw = true;
    }
    std::filesystem::remove(path);
    require(threw, "safetensors parser should reject offsets beyond payload");
}

void test_safetensors_rejects_bad_dtype_shape_size() {
    const std::filesystem::path path =
        temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
    const std::string header =
        R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,8]}})";
    write_safetensors_fixture(path, header, 8);

    bool threw = false;
    try {
        static_cast<void>(lcqi::load_safetensors_manifest(path));
    } catch (const std::runtime_error&) {
        threw = true;
    }
```

```
    std::filesystem::remove(path);
    require(threw, "safetensors parser should reject dtype/shape byte mismatch");
}

void test_reference_rms_norm() {
    const std::vector<float> input{3.0F, 4.0F};
    const std::vector<float> weight{1.0F, 0.5F};
    std::vector<float> output(2, 0.0F);
    lcqi::rms_norm(input, weight, 0.0F, output);
    require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
    require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");
}

void test_reference_rope() {
    std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
    lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
    require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
    require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
    require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine mismatch");
    require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
}

void test_reference_kv_attention() {
    lcqi::DecoderConfig config;
    config.hidden_size = 4;
    config.query_heads = 2;
    config.kv_heads = 1;
    config.head_dim = 2;
    config.intermediate_size = 4;
    config.vocab_size = 4;
    config.max_context = 4;

    lcqi::ReferenceKVCache cache(config, 1);
    const std::vector<float> key{1.0F, 0.0F};
    const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, LCQI_ATTENTION_EXPECTED_1};
    cache.append(0, 0, key, value);
    require(cache.filled_tokens() == 1, "KV filled token count mismatch");
    require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");

    const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
    std::vector<float> output(4, 0.0F);
    lcqi::attention_decode(config, cache, 0, 0, query, output);
    require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0 mismatch");
    require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1 mismatch");
    require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0 mismatch");
```

```
290     require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1 mismatch");
291 }
292
293 void test_reference_decoder_end_to_end() {
294     const lcqi::ReferenceDecoderModel model = lcqi::make_tiny_reference_decoder_model();
295     const std::vector<std::int32_t> tokens{1, 2, 3};
296     const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model, tokens);
297     require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,
298             "reference decoder logits size mismatch");
299     require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
300             "reference decoder predicted token mismatch");
301     for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ++index) {
302         require_close(result.logits[index],
303                       LCQI_REFERENCE_DECODER_LOGITS[index],
304                       "reference decoder golden logit mismatch");
305     }
306 }
307
308 void test_reference_decoder_trace_contract() {
309     const lcqi::ReferenceDecoderModel model = lcqi::make_tiny_reference_decoder_model();
310     const std::vector<std::int32_t> tokens{1, 2, 3};
311     const lcqi::ReferenceDecodeTraceResult trace =
312         lcqi::run_reference_decode_with_trace(model, tokens);
313
314     require(trace.result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
315             "reference trace predicted token mismatch");
316     require(!trace.checkpoints.empty(), "reference trace checkpoints missing");
317
318     const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
319     require(first.name == "embedding", "first checkpoint should be embedding");
320     require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
321             "first checkpoint token position mismatch");
322     require(first.dtype == "f32", "first checkpoint dtype mismatch");
323     require(first.layout == "row_major", "first checkpoint layout mismatch");
324     require(first.shape.size() == 1 &&
325                 first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
326             "first checkpoint shape mismatch");
327     require(first.stride.size() == 1 && first.stride[0] == 1,
328             "first checkpoint stride mismatch");
329
330     bool saw_logits = false;
331     bool saw_kv_slot = false;
332     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints) {
333         if (checkpoint.name == "kv_cache_key_slot") {
334             saw_kv_slot = true;
```

```
335             require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch");
336             require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout mismatch");
337         }
338         if (checkpoint.name == "logits") {
339             saw_logits = true;
340             require(checkpoint.token_position == LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
341                     "logits checkpoint token position mismatch");
342             require(checkpoint.shape.size() == 1 &&
343                         checkpoint.shape[0] ==
344                             static_cast<std::int32_t>(LCQI_REFERENCE_DECODER_VOCAB_SIZE),
345                     "logits checkpoint shape mismatch");
346             require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.size(),
347                     "logits checkpoint value size mismatch");
348             for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ++index) {
349                 require_close(checkpoint.values[index],
350                               LCQI_REFERENCE_DECODER_LOGITS[index],
351                               "trace logits checkpoint mismatch");
352             }
353         }
354     }
355     require(saw_kv_slot, "KV checkpoint missing");
356     require(saw_logits, "logits checkpoint missing");
357 }
358
359 void test_packed_i8_kernel_matches_scalar() {
360     for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
361         const lcqi::QuantizedLinearLayer layer =
362             lcqi::make_deterministic_i8_layer(shape_case.input_size,
363                                               shape_case.output_size,
364                                               0.03125F);
365         const lcqi::PackedLinearI8 packed =
366             lcqi::pack_linear_i8(layer, shape_case.output_block_size);
367         const std::vector<float> input = lcqi::make_deterministic_input(layer.input_size);
368         std::vector<float> scalar_output(static_cast<std::size_t>(layer.output_size), 0.0F);
369         std::vector<float> packed_output(static_cast<std::size_t>(layer.output_size), 0.0F);
370         lcqi::linear_i8(layer, input, scalar_output);
371         lcqi::linear_i8_packed(packed, input, packed_output);
372         require(lcqi::max_abs_diff(scalar_output, packed_output) <= LCQI_KERNEL_MAX_DIFF,
373                 "packed int8 kernel output differs from scalar");
374
375         const lcqi::PackedLinearI8 simd_packed =
376             lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
```

```
        if (lcqi::linear_i8_packed_avx2_available()) {
            std::vector<float> avx2_output(static_cast<std::size_t>(layer.output_size), 0.0F);
            lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
            require(lcqi::max_abs_diff(scalar_output, avx2_output) <= LCQI_KERNEL_MAX_DIFF,
                    "AVX2 int8 kernel output differs from scalar");
        }
    }
}

}  // namespace

int main() {
    try {
        test_tiny_mlp();
        test_tokenizer_contract();
        test_tokenizer_rejects_unknown_without_unk();
        test_safetensors_manifest_contract();
        test_safetensors_rejects_bad_offsets();
        test_safetensors_rejects_bad_dtype_shape_size();
        test_reference_rms_norm();
        test_reference_rope();
        test_reference_kv_attention();
        test_reference_decoder_end_to_end();
        test_reference_decoder_trace_contract();
        test_packed_i8_kernel_matches_scalar();

        std::cout << "lcqi tests passed\n";
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "lcqi test failure: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

Listing 16.25 models/tiny_mlp_i8.txt

```
LCQI_MODEL_V1
input_size 4
hidden_size 3
output_size 2
hidden_scale 0.1
hidden_weights
5 -3 2 1
-4 6 1 -2
3 2 -5 4
hidden_bias
0.1 -0.2 0.0
output_scale 0.05
output_weights
4 -6 2
```

```
-3 5 8
output_bias
-0.5 0.4
```

Listing 16.26 models/tiny_tokenizer.txt

```
LCQI_TOKENIZER_V1
bos_id 0
eos_id 4
unk_id -1
chat_template_hash tiny-chat-template-v1
vocab_size 5
vocab
0 <bos>
1 tok1
2 tok2
3 tok3
4 <eos>
```

术语表

术语	实践含义
manifest	模型资产清单，记录 tensor name、dtype、shape、offset、checksum 和 tokenizer/配置版本。
shape verifier	在 kernel 运行前检查 shape、dtype、layout、量化 descriptor 和 state edge 是否匹配。
reference decoder	以清楚正确为目标的 decoder 实现，用来生成 trace 和 golden。
oracle	比较候选实现与 reference 的判定器，可使用精确相等或误差界。
trace checkpoint	推理链路中的可复查节点，至少包含 shape、stride、dtype、layout、checksum 和摘要值。
KV cache	attention 历史 K/V 状态，跨 token 存活，不能被普通 activation planner 复用。
packing	把权重改写成更适合热循环和 SIMD 的布局。
backend	执行计划落到机器的实现，例如 scalar、AVX2、AVX-512、GPU 或库调用。
fallback	后端能力不满足时显式降级到可验证路径，并记录原因。
SLO	服务目标，例如 TTFT、TPOT、queue wait、拒绝率和取消回收时间。